

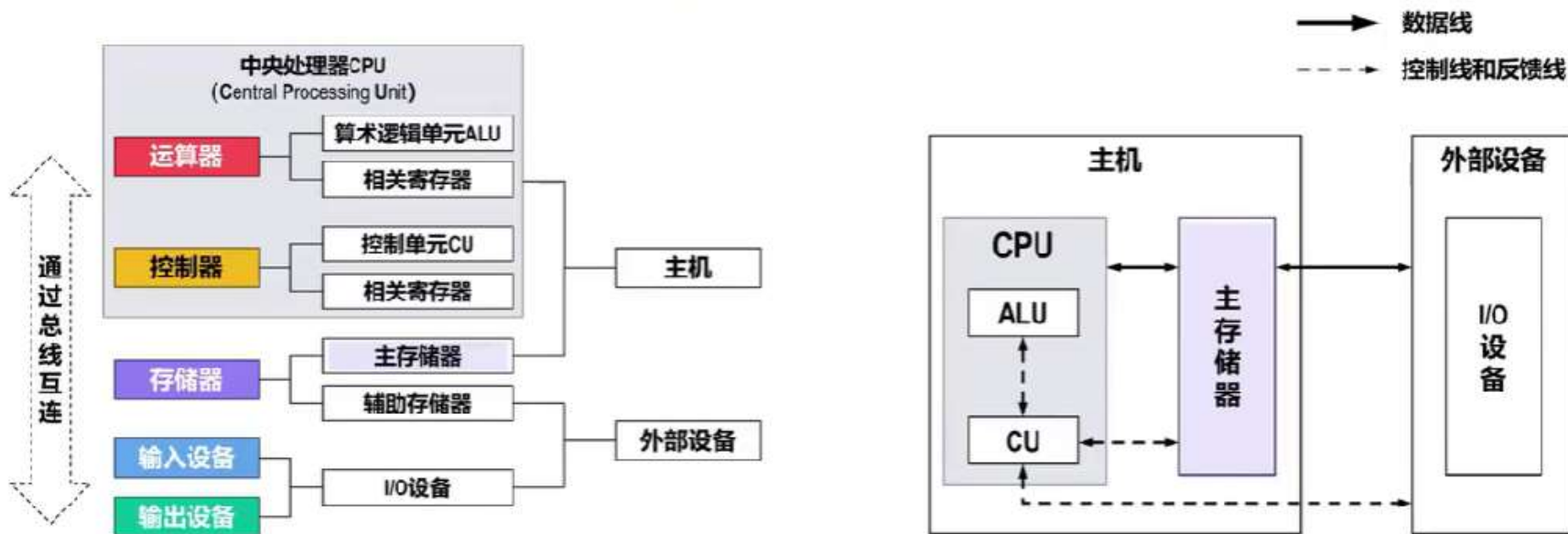
考点九：存储器

■ 现代计算机**以存储器为中心**，它是计算机中**存放指令和数据**的主要部件。

☐ 存储器的容量越大，能存储的信息越多。

☐ 提高存储系统的访问速度，是提高计算机处理信息速度的重要措施。

■ 因此，开发具有**大容量**、**高速度**和**低成本**的**存储系统**是计算机技术发展的关键目标之一。

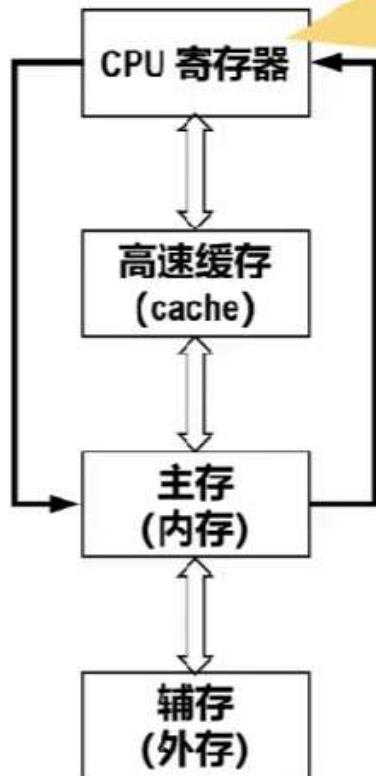


考点九：存储器



考点九：存储器

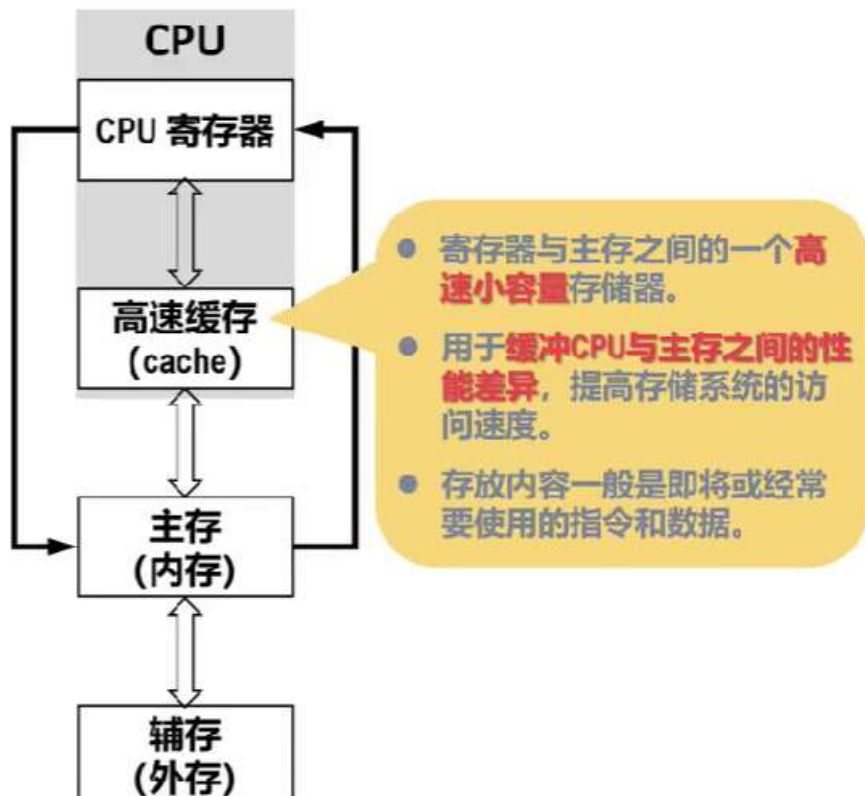
存储器——按功能分类



- CPU内部的多个寄存器（例如MAR、MDR、ACC、MQ等）。
- 用于存放地址、数据以及运算的中间结果。
- 速度与CPU匹配，容量极小。

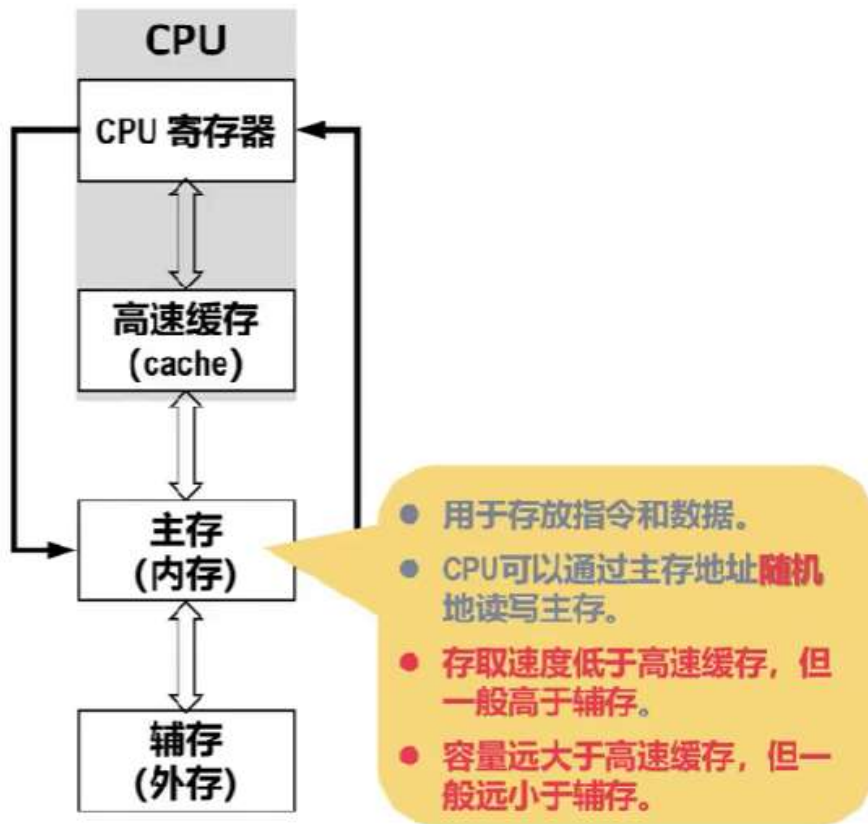
考点九：存储器

存储器——按功能分类



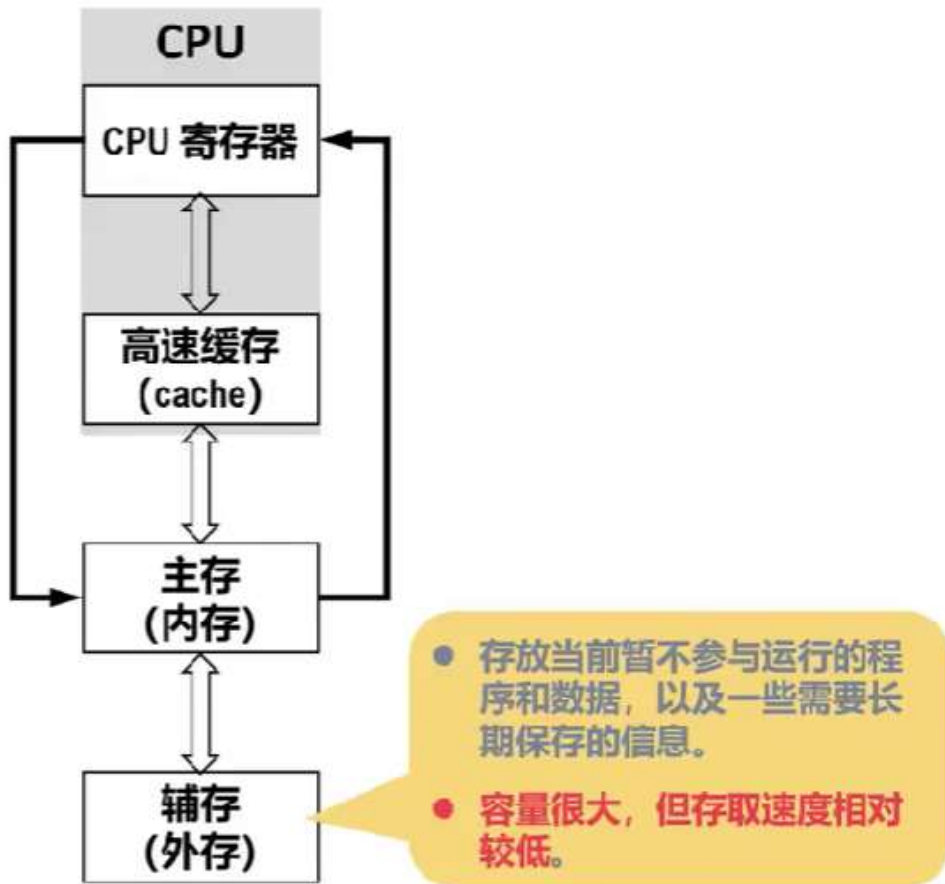
考点九：存储器

存储器——按功能分类



考点九：存储器

存储器——按功能分类

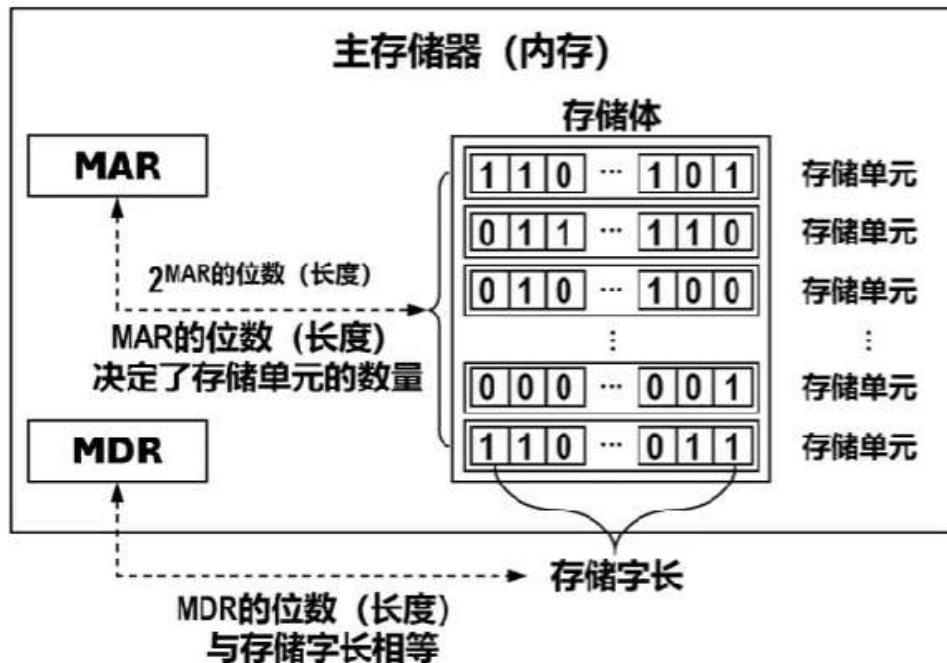


考点九：存储器

存储器性能指标——存储容量

■ 存储容量是指存储器可以**存储的二进制信息的总量**。

【举例】



$$\text{存储容量} = \text{存储字长} \times \text{存储单元数量}$$

考点九：存储器

存储器性能指标——存取速度

存取时间

- 启动一次存储器操作到该操作完成所经历的时间。

□ 读出时间
□ 写入时间

可能不同
例如闪存
读快写慢

存取周期

- 连续两次访问存储器操作（读操作或写操作）之间所需要的最短时间间隔。
- 对于主存，存取周期除包括存取时间外，还包括存储器状态的稳定恢复时间，因此存取周期略大于存取时间。



存储器带宽

- 单位时间内存储器所能传输的信息量（单位：b/s或B/s）。
 - 它是衡量数据传输速率的重要指标，与一次传输的数据位的多少和存取时间的长短有关。
- 一般而言，数据位宽越大、存取时间越短，则存储器带宽越高。

考点九：存储器

存储器层次结构

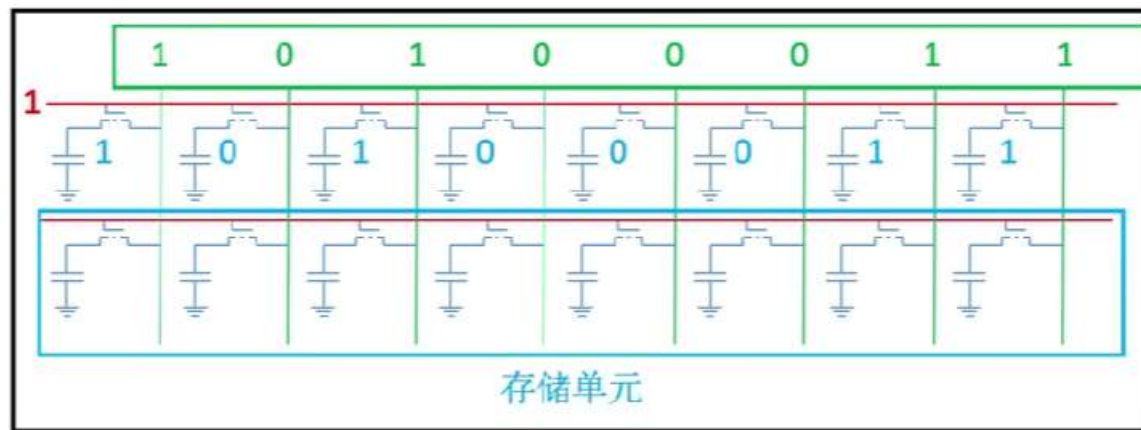
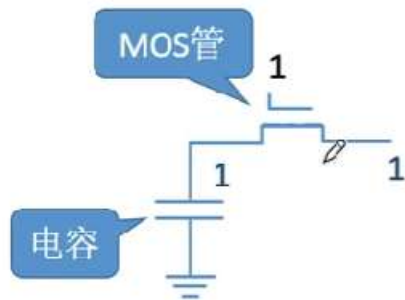
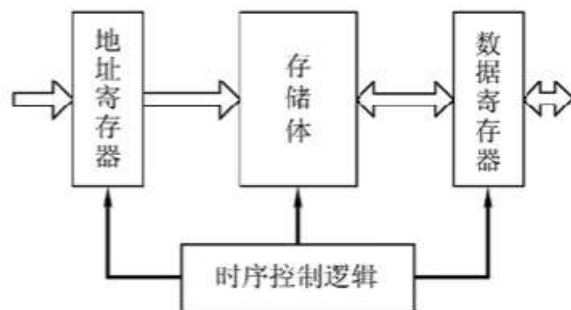


- **上层存储器可为下层存储器做缓冲**，将最经常使用的数据的副本调度到上层，使得CPU只需要访问上层的快速小容量存储器即可获得大部分数据。可以**有效提高存储系统的访问速度**，**缓解CPU与主存（内存）、主存（内存）与辅存（外存）的性能差异**。
- 另外，**使用大容量辅存（外存）**，**缓解了主存（内存）容量不足的问题**。

基于这种层次结构，就可构建出满足应用需求的 **存储容量大、存取速度快、成本低** 的存储系统。

考点九：存储器

主存储器——原理



存储字长: 8bit

字节: Byte

1Byte = 8bit

考点九：存储器

DRAM/SRAM

Static Random Access Memory

Dynamic Random Access Memory

类 型 特 点	SRAM（静态RAM）	DRAM（动态RAM）
存储信息	触发器	电容
破坏性读出	非	是
读出后需要重写？（再生）	不用	需要
运行速度	快	慢
集成度	低	高
发热量	大	小
存储成本	高	低
易失/非易失性存储器？	易失（断电后信息消失）	易失（断电后信息消失）
需要“刷新”？	不需要	需要
送行列地址	同时送	分两次送

常用作Cache

常用作主存

考点九：存储器

ROM

MROM (Mask Read-Only Memory) ——掩模式只读存储器

厂家按照客户需求，在芯片生产过程中直接写入信息，之后**任何人不可重写**（只能读出）
可靠性高、灵活性差、生产周期长、只适合批量定制

PROM (Programmable Read-Only Memory) ——可编程只读存储器

用户可用专门的PROM写入器写入信息，写**一次之后就不可更改**

EPROM (Erasable Programmable Read-Only Memory) ——可擦除可编程只读存储器

允许用户写入信息，之后用某种方法擦除数据，**可进行多次重写**

UVEPROM (ultraviolet rays) ——用紫外线照射8~20分钟，擦除**所有**信息

EEPROM (也常记为E²PROM, 第一个E是Electrically) ——可用“电擦除”的方式，擦除**特定的字**

Flash Memory ——闪速存储器（注：U盘、SD卡就是闪存）

每个存储元只需单个MOS管，位密度比RAM高

在EEPROM基础上发展而来，断电后也能保存信息，且**可进行多次快速擦除重写**

注意：由于闪存需要先擦除在写入，因此**闪存的“写”速度要比“读”速度更慢。**

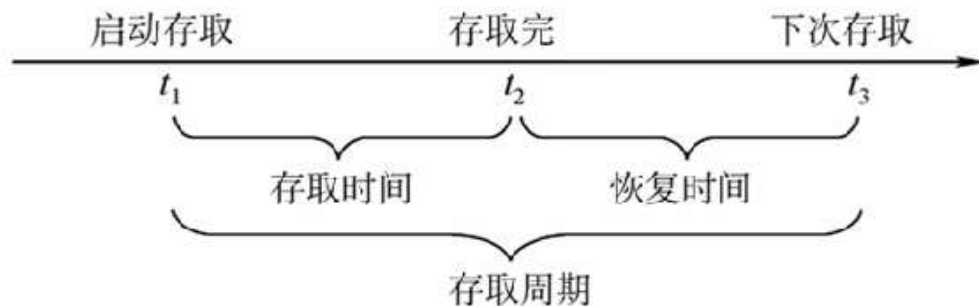
拓：手机辅存也使用Flash芯片，但相比SSD使用的芯片集成度高、功耗低、价格贵

SSD (Solid State Drives) ——固态硬盘

由控制单元+存储单元（Flash 芯片）构成，与闪速存储器的核心区别在于控制单元不一样，但存储介质都类似，**可进行多次快速擦除重写**。SSD速度快、功耗低、价格高。目前个人电脑上常用SSD取代传统的机械硬盘

考点九：存储器

内存优化



存取周期：可以连续读/写的最短时间间隔

注：DRAM芯片的恢复时间比较长，有可能是存取时间的几倍（SRAM的恢复时间较短）

如：存取时间为 r ，存取周期为 T ， $T=4r$



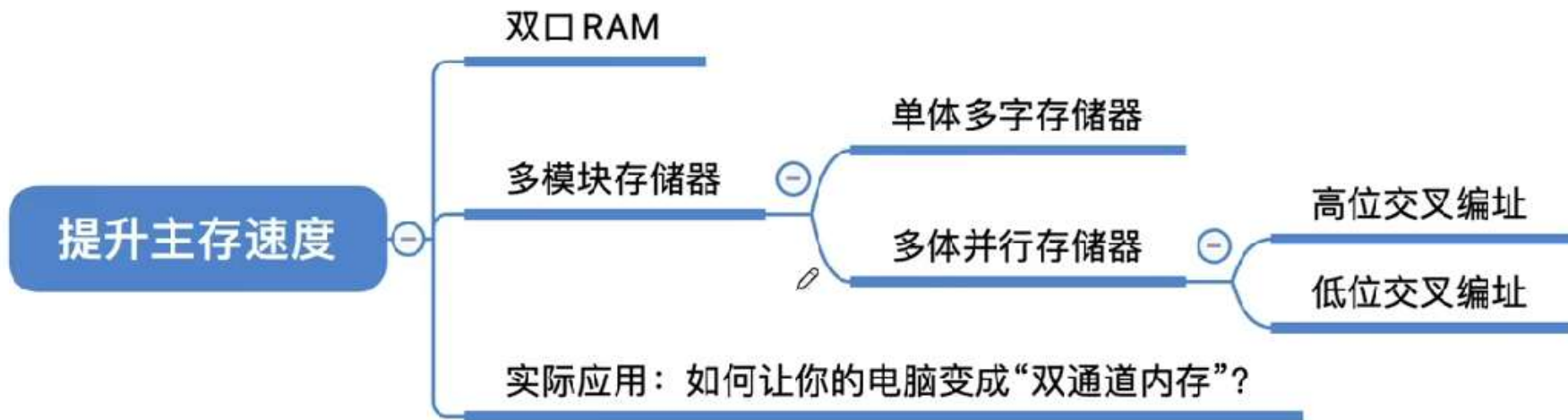
我能怎么办

多核CPU都要访存，怎么办？

CPU的读写速度比主存快很多，
主存恢复时间太长怎么办？

考点九：存储器

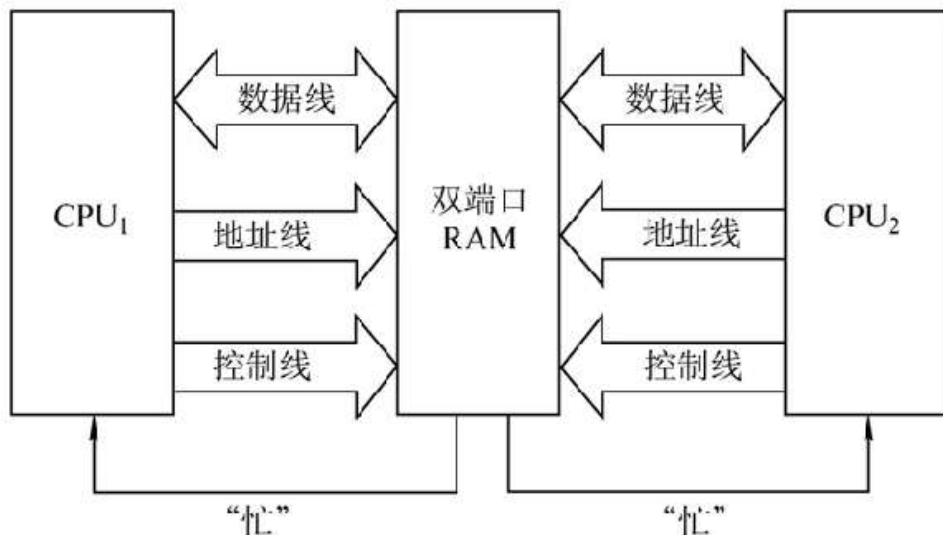
内存优化



考点九：存储器

内存优化——双端口RAM

需要有两组完全独立的数据线、地址线、控制线。CPU、RAM中也要有更复杂的控制电路



作用：优化多核CPU访问一根内存条的速度

解决方法：置“忙”信号为0，由判断逻辑决定暂时关闭一个端口（即被延时），未被关闭的端口正常访问，被关闭的端口延长一个很短的时间段后再访问。

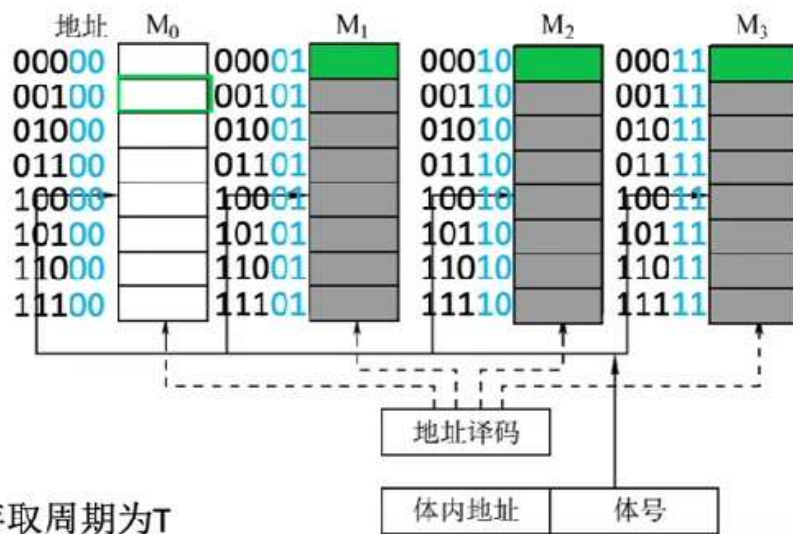
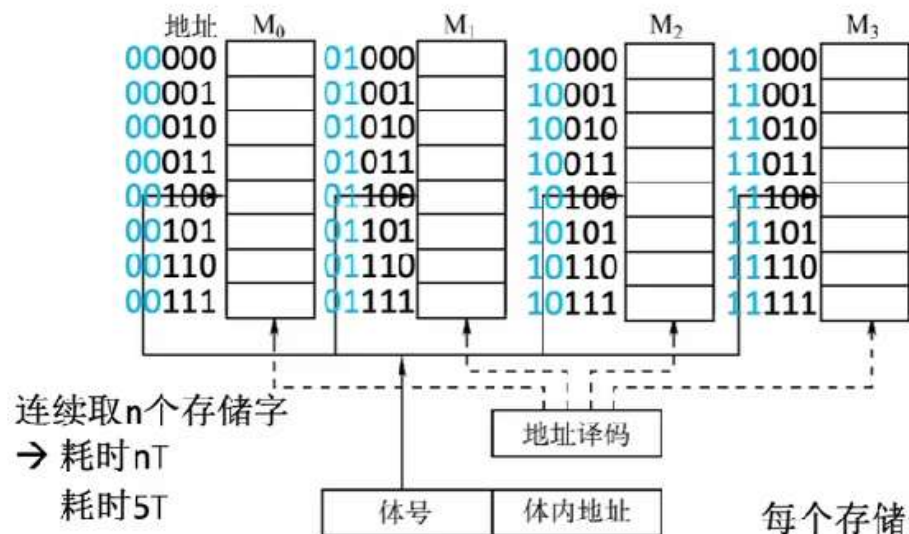
两个端口对同一主存操作有以下4种情况：

1. 两个端口同时对不同的地址单元存取数据。😊
2. 两个端口同时对同一地址单元读出数据。😊
3. 两个端口同时对同一地址单元写入数据。⊗写入错误
4. 两个端口同时对同一地址单元，一个写入数据，另一个读出数据。⊗读出错误

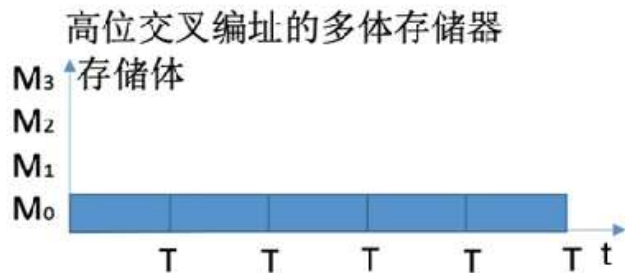
对比操作系统
“读者-写者问题”

考点九：存储器

内存优化——多模块存储

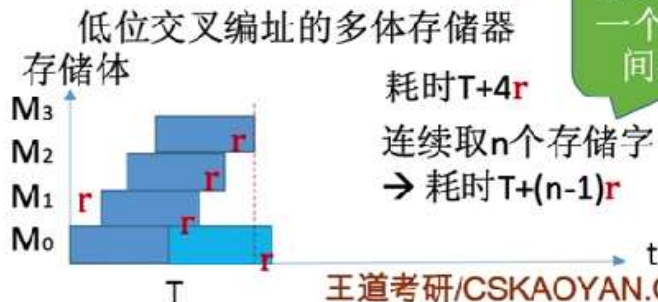


每个存储体存取周期为T
存取时间为r, 假设 $T=4r$



连续访问:

00000
00001
00010
00011
00100



宏观上读写
一个字的时间
接近 r

考点九：存储器

内存优化——多模块存储

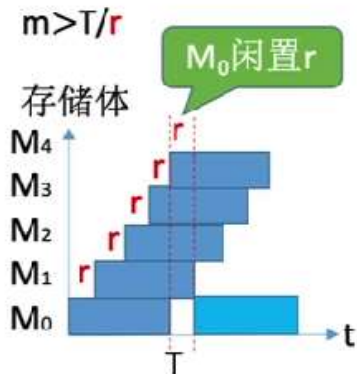
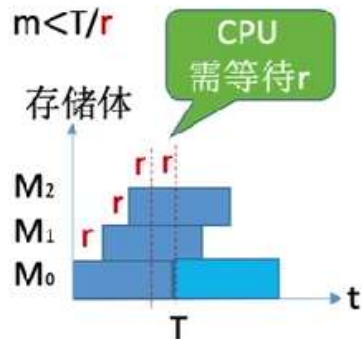
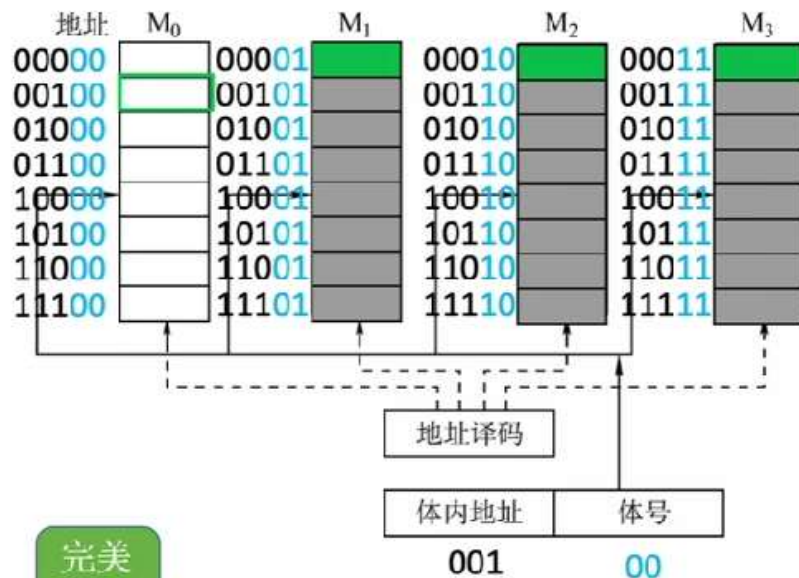
采用“**流水线**”的方式并行存取（宏观上并行，微观上串行）

宏观上，一个存储周期内， m 体交叉存储器可以提供的数据量为单个模块的 m 倍。

两种
常见
描述

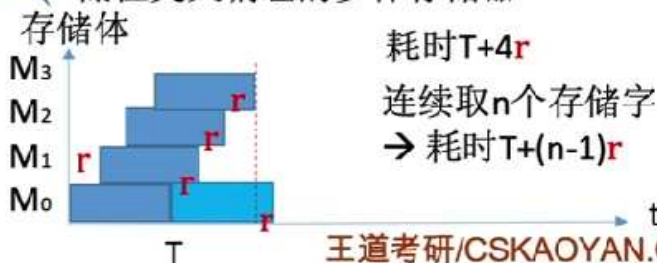
存取周期为 T ，**存取时间**为 r ，为了使流水线不间断，应保证模块数 $m \geq T/r$

存取周期为 T ，**总线传输周期**为 r ，为了使流水线不间断，应保证模块数 $m \geq T/r$



完美
衔接

低位交叉编址的多体存储器



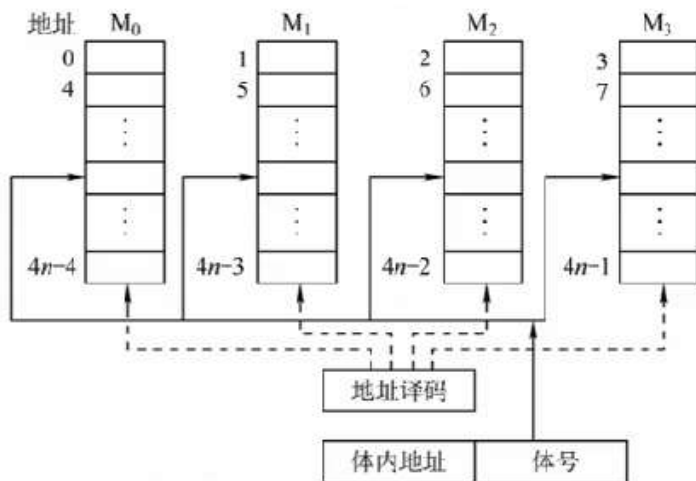
耗时 $T + 4r$

连续取 n 个存储字

→ 耗时 $T + (n-1)r$

考点九：存储器

内存优化——多模块存储



多体并行存储器

每个模块都有相同的容量和存取速度。
各模块都有独立的读写控制电路、地址寄存器和数据寄存器。
它们既能并行工作，又能交叉工作。

⋮			

单体多字存储器

每个存储单元存储 m 个字
总线宽度也为 m 个字
一次并行读出 m 个字

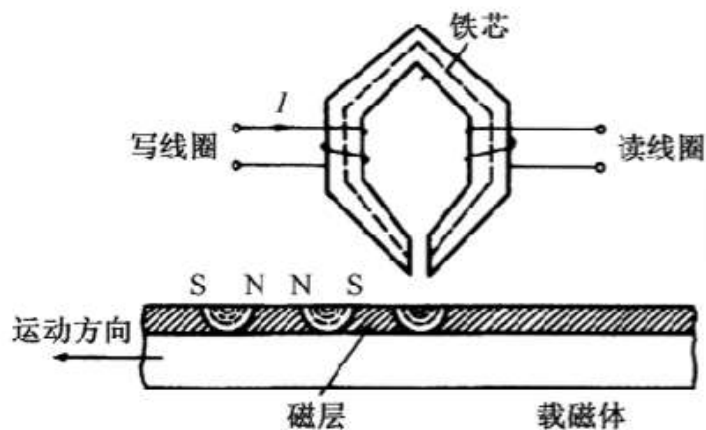
每次只能同时取 m 个字，不能单独取其中某个字
⑧指令和数据在主存内必须是连续存放的

考点九：存储器

外存储——磁盘

计算机的外存储器又称为辅助存储器，目前主要使用磁表面存储器。

所谓“磁表面存储”，是指把某些磁性材料薄薄地涂在金属铝或塑料表面上作为载磁体来存储信息。磁盘存储器、磁带存储器和磁鼓存储器均属于磁表面存储器。



磁表面存储器的优点：

- ①存储容量大，位价格低；
- ②记录介质可以重复使用；
- ③记录信息可以长期保存而不丢失，甚至可以脱机存档；
- ④非破坏性读出，读出时不需要再生。

磁表面存储器的缺点：

- ①存取速度慢；
- ②机械结构复杂；
- ③对工作环境要求较高。

外存储器既可以作为输入设备，也可以作为输出设备。
(既可以存数据，也可以读数据)

考点九：存储器

外存储——磁盘

1. 磁盘设备的组成

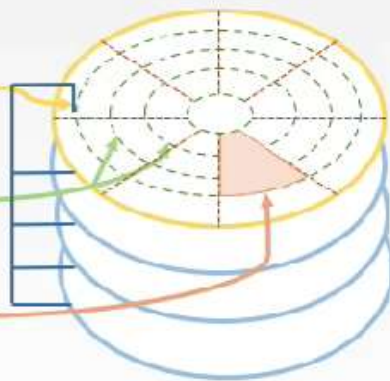
① 存储区域

一块硬盘含有若干个记录面，每个记录面划分为若干条磁道，而每条磁道又划分为若干个扇区，扇区（也称块）是磁盘读写的最小单位，也就是说磁盘按块存取。

磁头数 (Heads) 即记录面数，表示硬盘总共有多少个磁头，磁头用于读取/写入盘片上记录面的信息，一个记录面对应一个磁头。

柱面数 (Cylinders) 表示硬盘每一面盘片上有多少条磁道。在一个盘组中，不同记录面的相同编号（位置）的诸磁道构成一个圆柱面。

扇区数 (Sectors) 表示每一条磁道上有多少个扇区。



② 硬盘存储器

硬盘存储器由磁盘驱动器、磁盘控制器和盘片组成。

磁盘驱动器：核心部件是磁头组件和盘片组件，温彻斯特盘是一种可移动头固定盘片的硬盘存储器。

磁盘控制器：是硬盘存储器和主机的接口，主流的标准有IDE、SCSI、SATA等。

考点九：存储器

外存储——磁盘

2.磁盘的性能指标

① 磁盘的容量：一个磁盘所能存储的字节总数称为磁盘容量。磁盘容量有非格式化容量和格式化容量之分。

非格式化容量是指磁记录表面可以利用的磁化单元总数。

格式化容量是指按照某种特定的记录格式所能存储信息的总量。

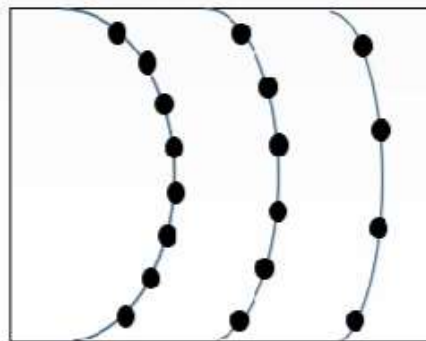
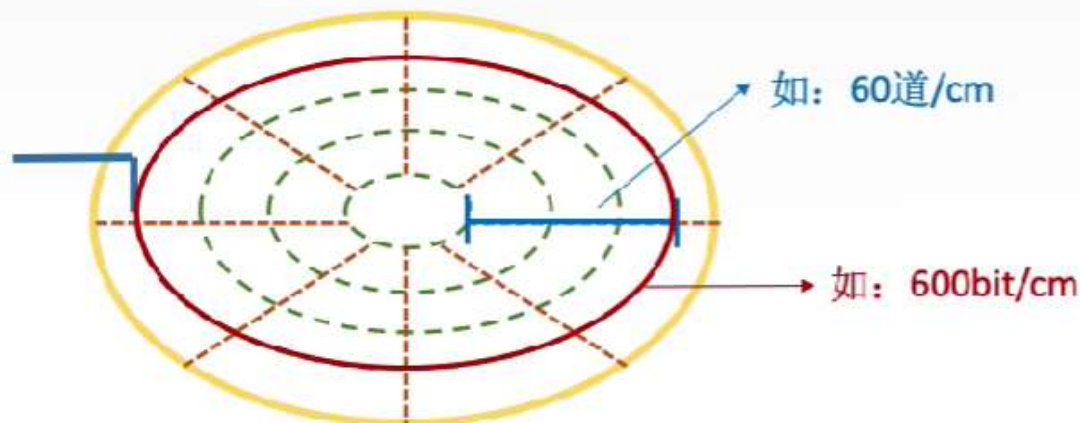
② 记录密度：记录密度是指盘片单位面积上记录的二进制的信息量，通常以道密度、位密度和面密度表示。

道密度是沿磁盘半径方向单位长度上的磁道数；

位密度是磁道单位长度上能记录的二进制代码位数；

面密度是位密度和道密度的乘积。

注意：磁盘所有磁道记录的信息量一定是相等的，并不是圆越大信息越多，故每个磁道的位密度都不同。



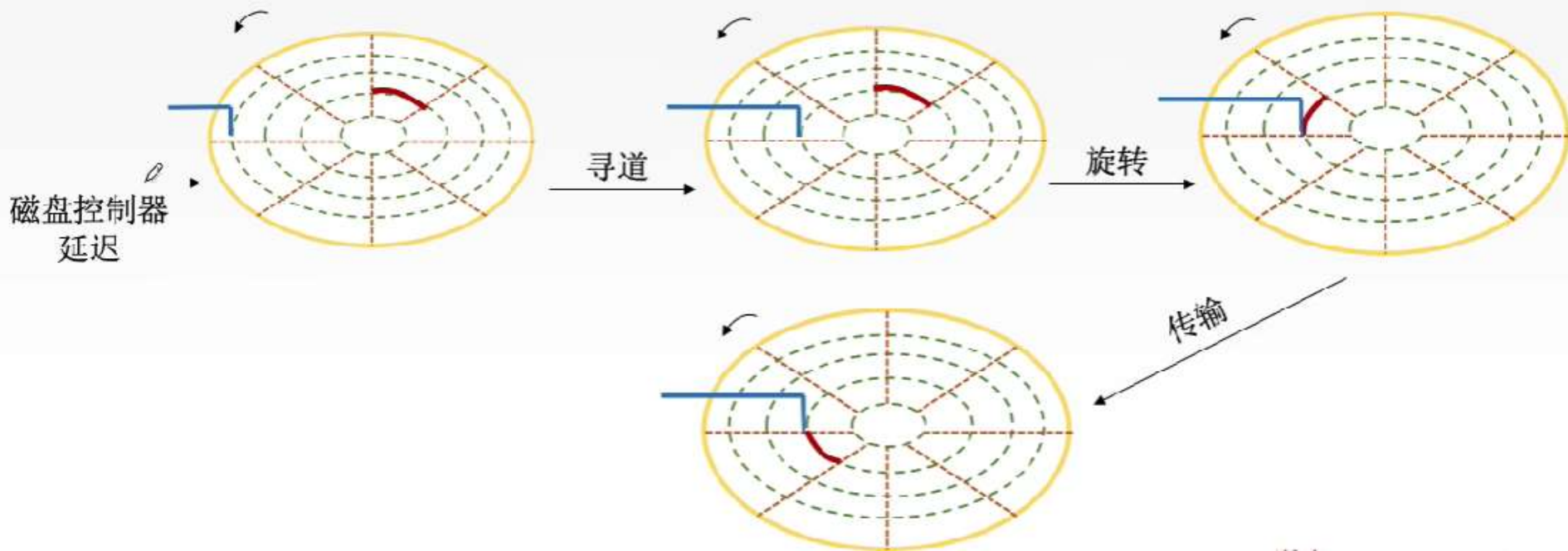
越内侧的磁道位密度越大

考点九：存储器

外存储——磁盘

2. 磁盘的性能指标

③ 平均存取时间： 平均存取时间 = 寻道时间（磁头移动到目的磁道）+ 旋转延迟时间（磁头定位到所在扇区）+ 传输时间（传输数据所花费的时间）



考点九：存储器

外存储——磁盘

2.磁盘的性能指标

① 磁盘的容量：一个磁盘所能存储的字节总数称为磁盘容量。磁盘容量有非格式化容量和格式化容量之分。

非格式化容量是指磁记录表面可以利用的磁化单元总数。

格式化容量是指按照某种特定的记录格式所能存储信息的总量。

② 记录密度：记录密度是指盘片单位面积上记录的二进制的信息量，通常以道密度、位密度和面密度表示。

道密度是沿磁盘半径方向单位长度上的磁道数；

位密度是磁道单位长度上能记录的二进制代码位数；

面密度是位密度和道密度的乘积。

注意：磁盘所有磁道记录的信息量一定是相等的，并不是圆越大信息越多，故每个磁道的位密度都不同。

③ 平均存取时间：

平均存取时间 = 寻道时间（磁头移动到目的磁道）+
旋转延迟时间（磁头定位到所在扇区）+
传输时间（传输数据所花费的时间）

④ 数据传输率：磁盘存储器在单位时间内向主机传送数据的字节数，称为数据传输率。

假设磁盘转数为 r （转/秒），每条磁道容量为 N 个字节，则数据传输率为 $D_T=rN$

考点九：存储器

外存储——磁盘

3. 磁盘地址

主机向磁盘控制器发送寻址信息，磁盘的地址一般如图所示：

驱动器号	柱面（磁道）号	盘面号	扇区号
------	---------	-----	-----

一台电脑可能有多个硬盘

移动磁头臂（寻道）

激活某个磁头

通过旋转将特定扇区划过磁头下方

若系统中有4个驱动器，每个驱动器带一个磁盘，每个磁盘256个磁道、16个盘面，每个盘面划分为16个扇区，则每个扇区地址要18位二进制代码：

驱动器号（2bit）	柱面（磁道）号（8bit）	盘面号（4bit）	扇区号（4bit）
------------	---------------	-----------	-----------

4. 硬盘的工作过程

硬盘的主要操作是寻址、读盘、写盘。每个操作都对应一个控制字，硬盘工作时，第一步是取控制字，第二步是执行控制字。

硬盘属于机械式部件，其读写操作是串行的，不可能在同一时刻既读又写，也不可能在同一时刻读两组数据或写两组数据。

考点九：存储器

磁盘阵列

RAID (Redundant Array of Inexpensive Disks, 廉价冗余磁盘阵列) 是将多个独立的物理磁盘组成一个独立的逻辑盘，数据在多个物理盘上分割交叉存储、并行访问，具有更好的存储性能、可靠性和安全性。

RAID的分级如下所示。在RAID1~RAID5的几种方案中，无论何时磁盘损坏，都可以随时拔出受损的磁盘再插入好的磁盘，而数据不会损坏。

RAID0: 无冗余和无校验的磁盘阵列。

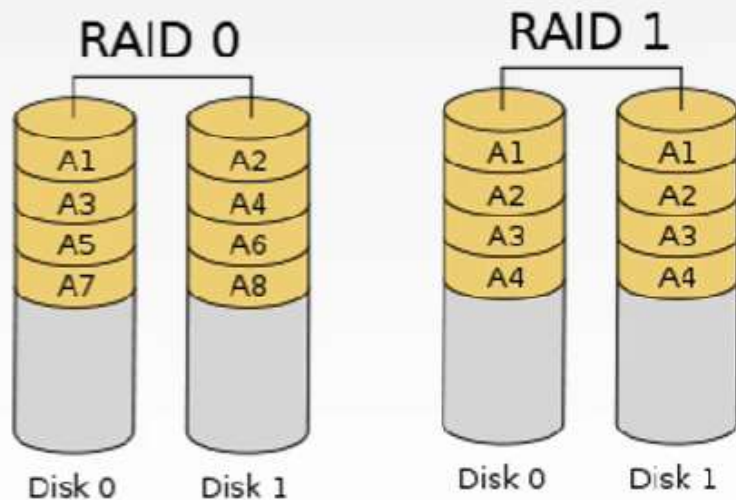
RAID1: 镜像磁盘阵列。

RAID2: 采用纠错的海明码的磁盘阵列。

RAID3: 位交叉奇偶校验的磁盘阵列。

RAID4: 块交叉奇偶校验的磁盘阵列。

RAID5: 无独立校验的奇偶校验磁盘阵列。



RAID0: 逻辑上相邻的两个扇区在物理上存到两个磁盘，类比第三章“低位交叉编址的多体存储器”

RAID1: 很粗暴，存两份数据

考点九：存储器

磁盘阵列

RAID (Redundant Array of Inexpensive Disks, 廉价冗余磁盘阵列) 是将多个独立的物理磁盘组成一个独立的逻辑盘，数据在多个物理盘上分割交叉存储、并行访问，具有更好的存储性能、可靠性和安全性。

RAID的分级如下所示。在RAID1~RAID5的几种方案中，无论何时磁盘损坏，都可以随时拔出受损的磁盘再插入好的磁盘，而数据不会损坏。

RAID0: 无冗余和无校验的磁盘阵列。

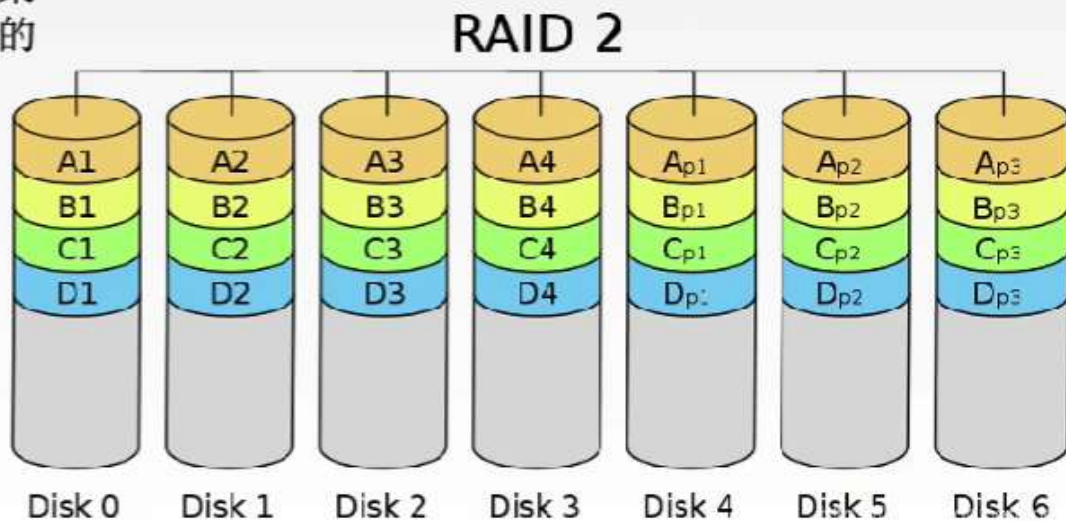
RAID1: 镜像磁盘阵列。

RAID2: 采用纠错的海明码的磁盘阵列。

RAID3: 位交叉奇偶校验的磁盘阵列。

RAID4: 块交叉奇偶校验的磁盘阵列。

RAID5: 无独立校验的奇偶校验磁盘阵列。



RAID2: 逻辑上连续的几个bit物理上分散存储在各个盘中
4bit信息位+3bit海明校验位——可纠正一位错

考点九：存储器

磁盘阵列

RAID (Redundant Array of Inexpensive Disks, 廉价冗余磁盘阵列) 是将多个独立的物理磁盘组成一个独立的逻辑盘, 数据在多个物理盘上分割交叉存储、并行访问, 具有更好的存储性能、可靠性和安全性。

RAID的分级如下所示。在RAID1~RAID5的几种方案中, 无论何时磁盘损坏, 都可以随时拔出受损的磁盘再插入好的磁盘, 而数据不会损坏。

RAID0: 无冗余和无校验的磁盘阵列

RAID0把连续多个数据块交替地存放在不同物理磁盘的扇区中, 几个磁盘交叉并行读写, 不仅扩大了存储容量, 而且提高了磁盘数据存取速度, 但RAID0没有容错能力。

RAID1: 镜像磁盘阵列。

RAID1是为了提高可靠性, 使两个磁盘同时进行读写, 互为备份, 如果一个磁盘出现故障, 可从另一磁盘中读出数据。两个磁盘当一个磁盘使用, 意味着容量减少一半。

RAID2: 采用纠错的海明码的磁盘阵列。

RAID3: 位交叉奇偶校验的磁盘阵列。

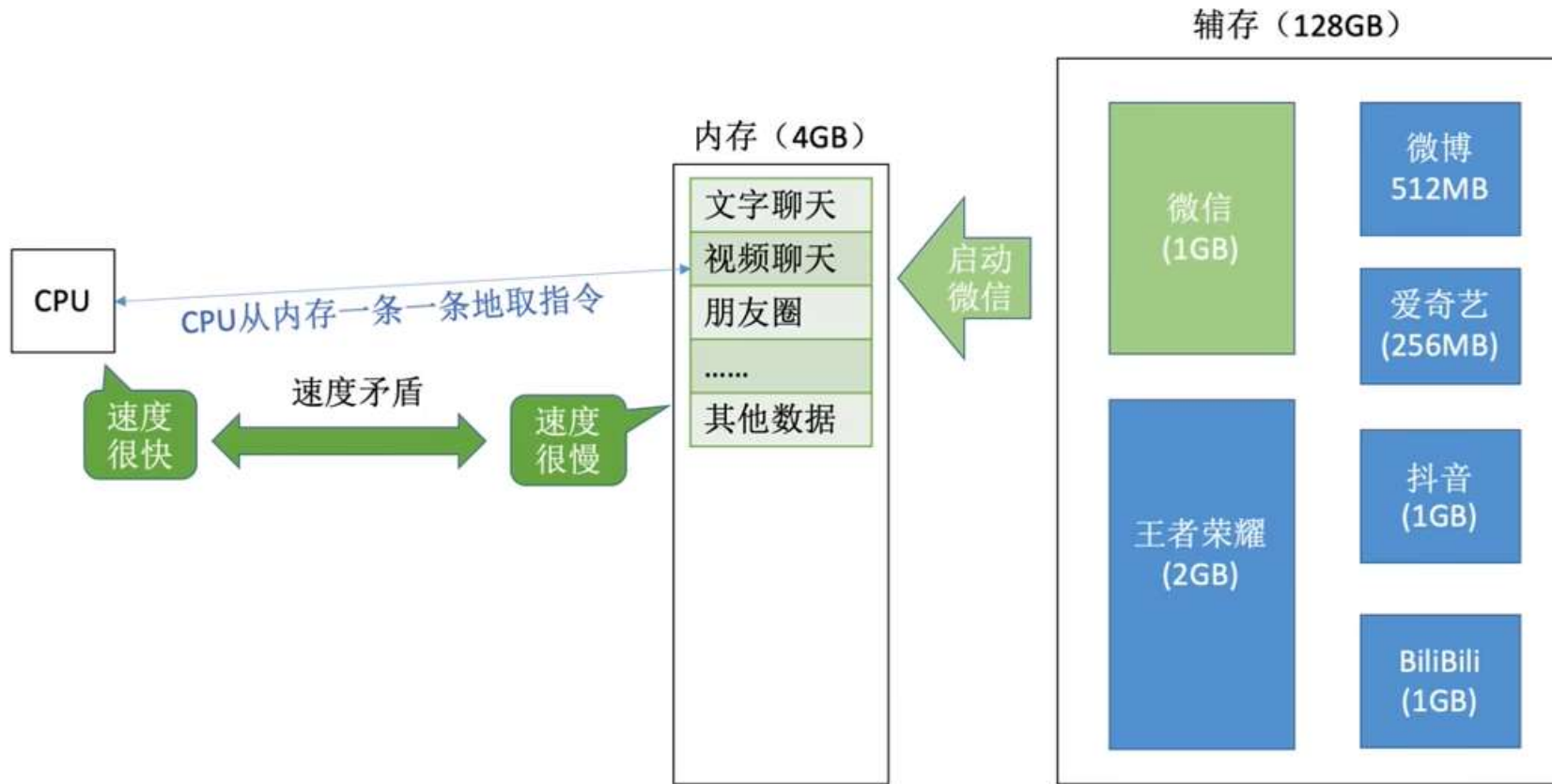
RAID4: 块交叉奇偶校验的磁盘阵列。

RAID5: 无独立校验的奇偶校验磁盘阵列。

RAID通过同时使用多个磁盘, 提高了传输率; 通过在多个磁盘上并行存取来大幅提高存储系统的数据吞吐量; 通过镜像功能, 可以提高安全可靠; 通过数据校验, 可以提供容错能力。

考点九：存储器

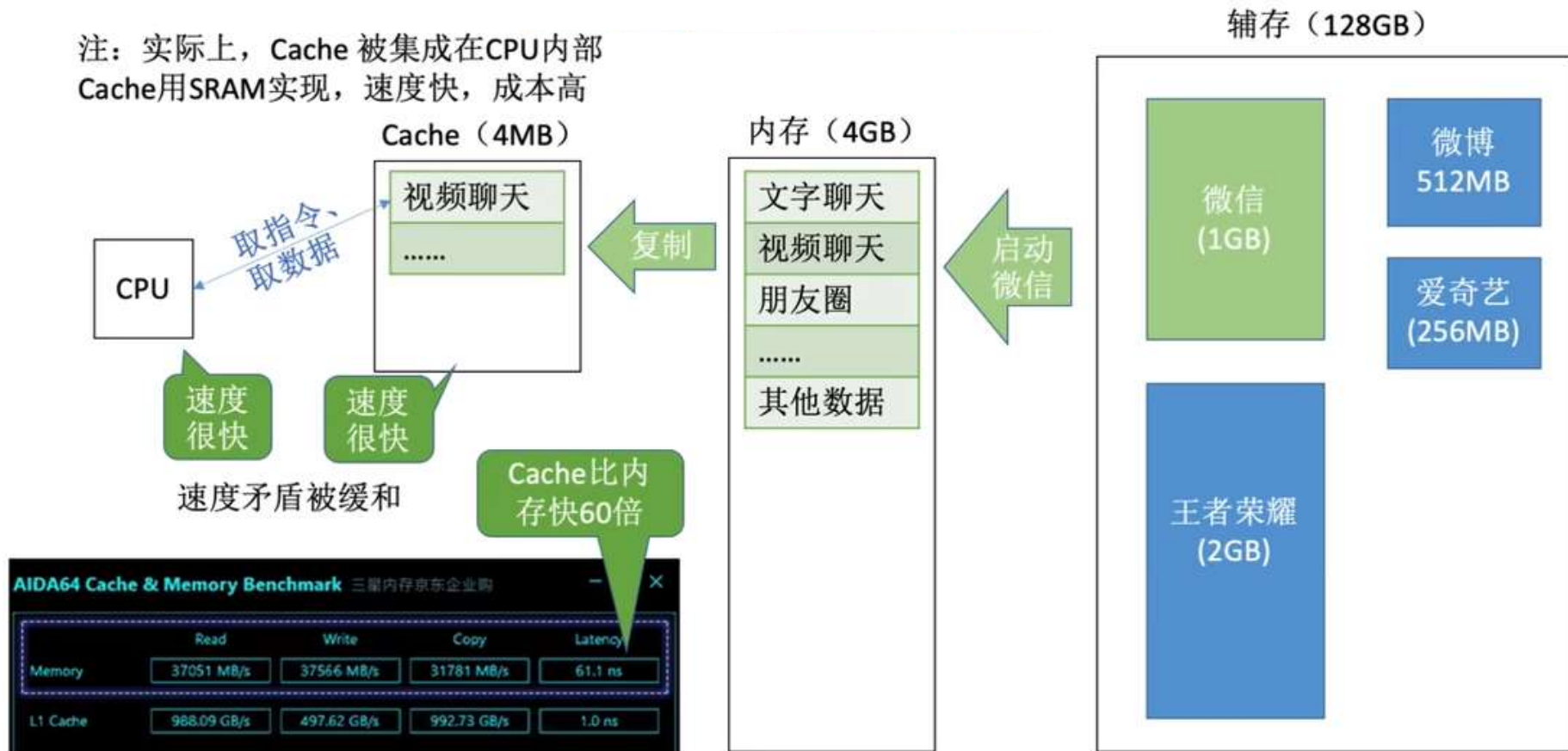
Cache



考点九：存储器

Cache

注：实际上，Cache 被集成在CPU内部
Cache用SRAM实现，速度快，成本高



考点九：存储器

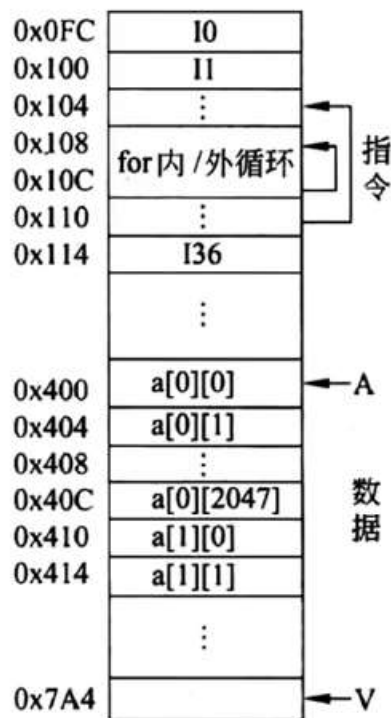
Cache——局部性原理

程序A:

```
1  int sumarrayrows(int a[M][N])
2  {
3      int i, j, sum = 0;
4      for (i = 0; i < M; i++)
5          for (j = 0; j < N; j++)
6              sum += a[i][j];
7      return sum;
8  }
```

程序B:

```
1  int sumarraycols(int a[M][N])
2  {
3      int i, j, sum = 0;
4      for (j = 0; j < N; j++)
5          for (i = 0; i < M; i++)
6              sum += a[i][j];
7      return sum;
8  }
```



指令和数据在内存中的存储

Eg: 数组元素、顺序执行的指令代码

空间局部性: 在最近的未来要用到的信息(指令和数据), 很可能与现在正在使用的信息在存储空间上是邻近的

时间局部性: 在最近的未来要用到的信息, 很可能是现在正在使用的信息

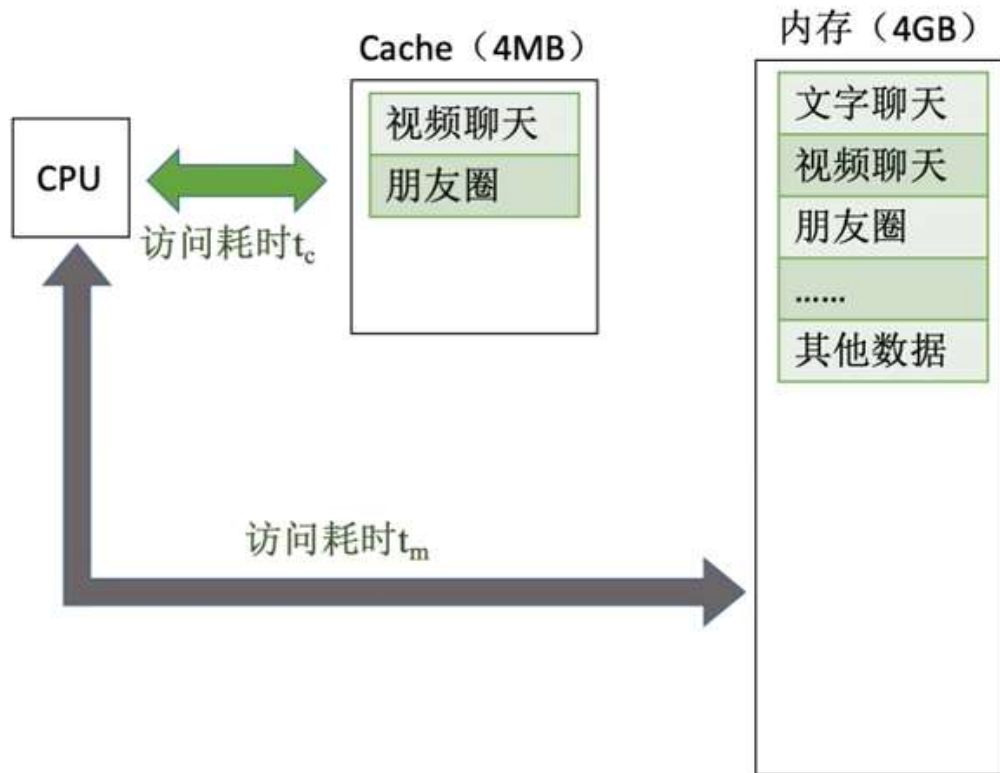
Eg: 循环结构的指令代码

基于局部性原理, 不难想到, 可以把CPU目前访问的地址“周围”的部分数据放到Cache中

程序B 按“列优先”访问二维数组, 空间局部性更差

考点九：存储器

Cache——局部性原理



设 t_c 为访问一次Cache所需时间， t_m 为访问一次主存所需时间

命中率H：CPU欲访问的信息已在Cache中的比率

缺失（未命中）率M = $1 - H$

Cache—主存 系统的**平均访问时间 t** 为

$$t = Ht_c + (1 - H)(t_c + t_m)$$

先访问Cache，若Cache未命中再访问主存

或 $t = Ht_c + (1 - H)t_m$

同时访问 Cache和主存，若Cache命中则立即停止访问主存

考点九：存储器

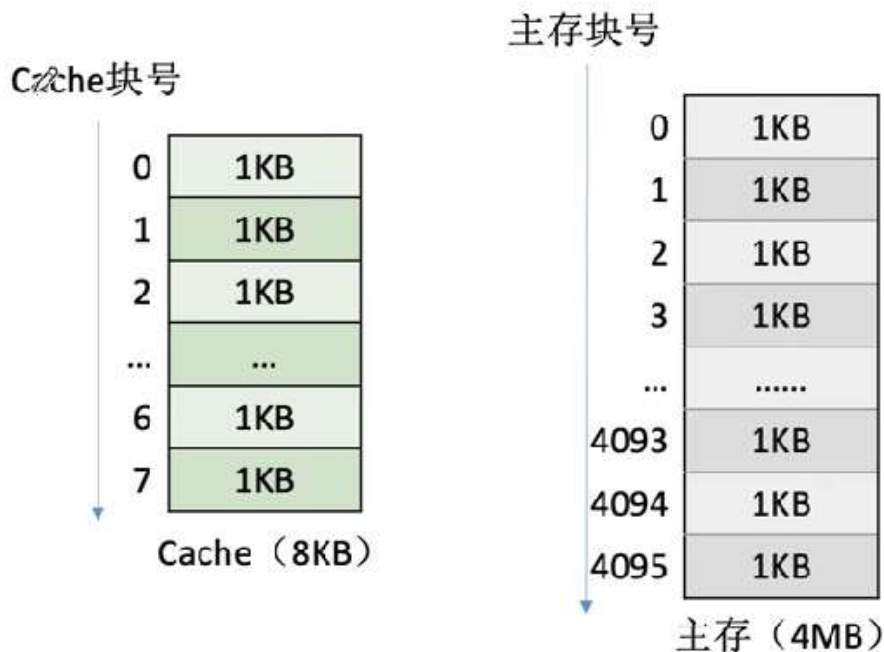
Cache——局部性原理

基于局部性原理，不难想到，可以把CPU目前访问的地址“**周围**”的部分数据放到Cache中。如何界定“**周围**”？

将主存的存储空间“**分块**”，如：每1KB为一块。主存与Cache之间以“**块**”为单位进行数据交换

注：操作系统中，通常将主存中的“一个**块**”也称为“一个**页/页面/页框**”

Cache中的“**块**”也称为“**行**”



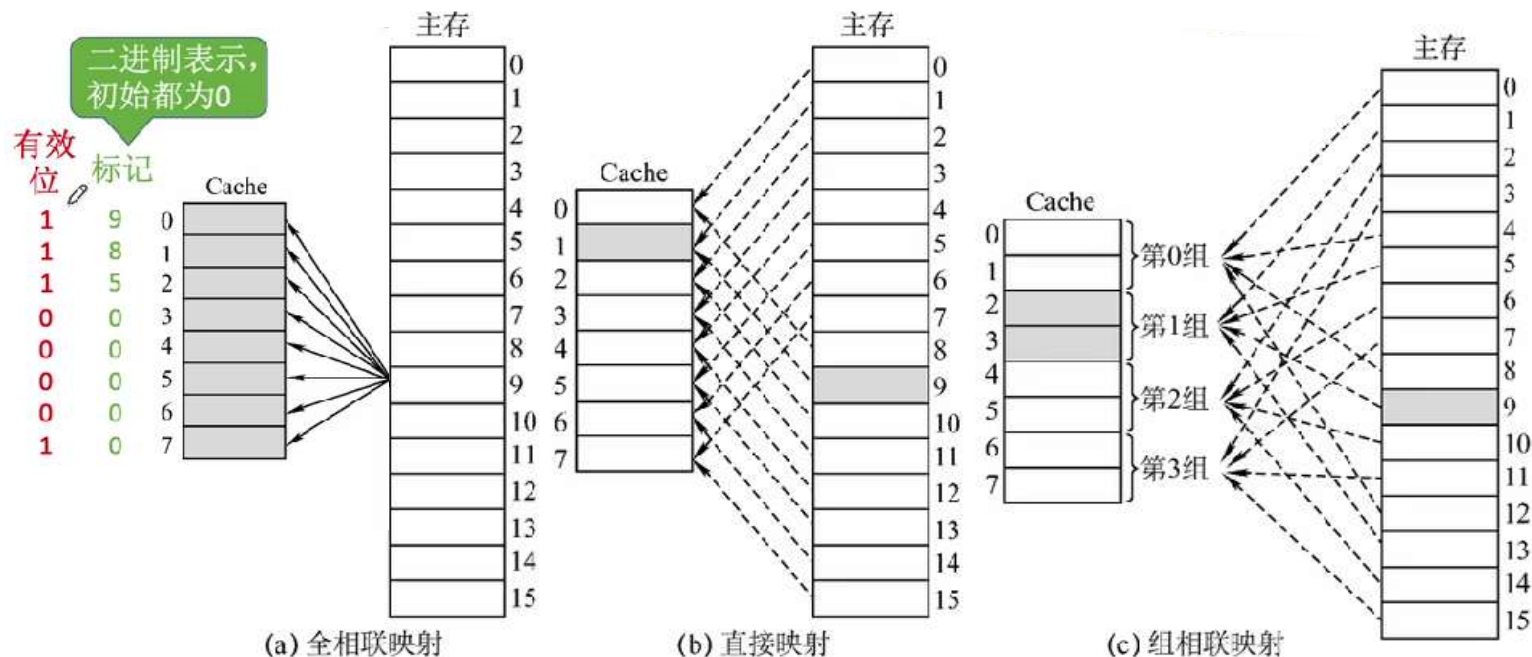
主存的地址共22位：

块号	块内地址
12位	10位

$4M=2^{22}$, $1K=2^{10}$
整个主存被分为 $2^{12} = 4096$ 块

考点九：存储器

Cache——与Memory的映射关系



主存块可以放在
Cache的任意位置

每个主存块只能放到一个特定的位置：
 $\text{Cache块号} = \text{主存块号} \% \text{Cache总块数}$

Cache块分为若干组，每个主存块可放到特定分组中的任意一个位置
 $\text{组号} = \text{主存块号} \% \text{分组数}$

考点九：存储器

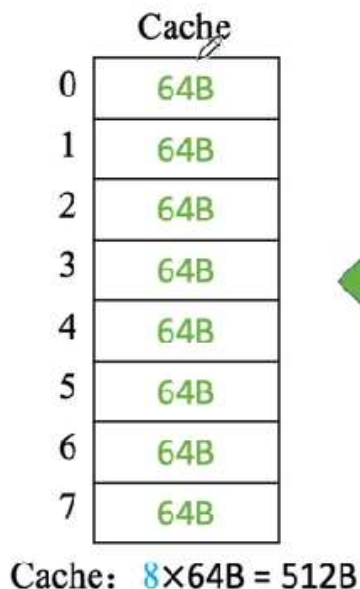
Cache——全相联映射

假设某个计算机的主存地址空间大小为256MB，按字节编址，其数据Cache有8个Cache行，行长为64B。

即Cache块，与主存块的大小相等

$256\text{M}=2^{28}$ 主存的地址共28位：

主存块号	块内地址
22位	6位



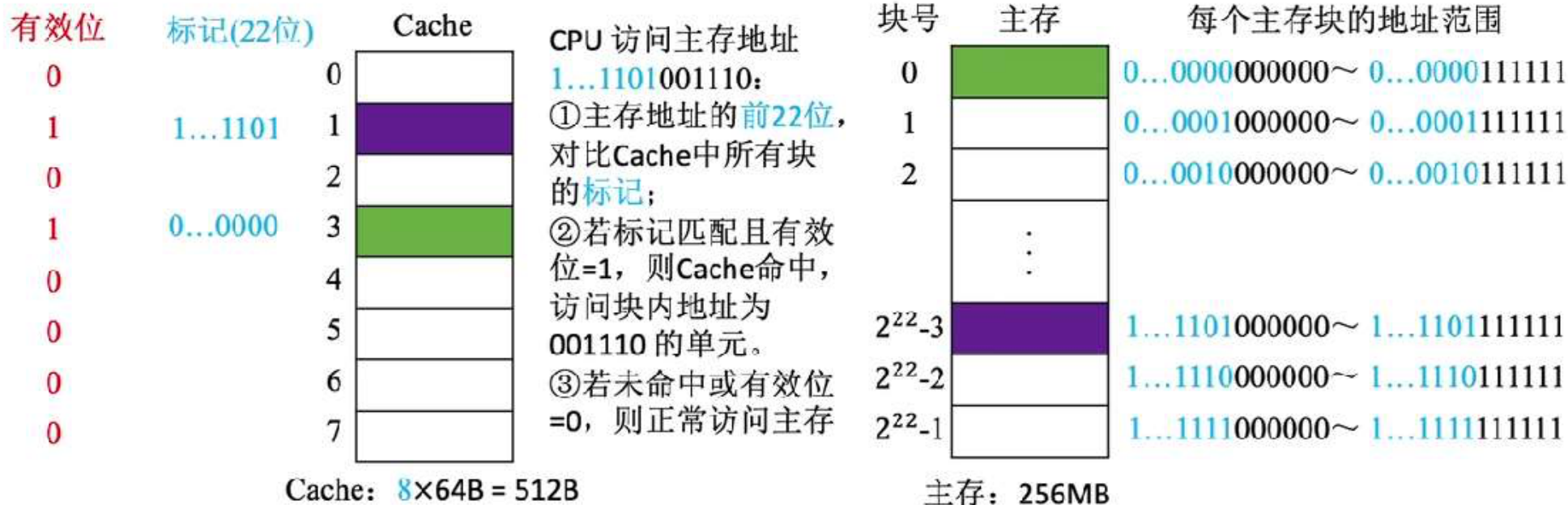
考点九：存储器 Cache——全相联映射

假设某个计算机的主存地址空间大小为256MB，按字节编址，其数据Cache有8个Cache行，行长为64B。

即Cache块，与主存块的大小相等

$256\text{M}=2^{28}$ 主存的地址共28位：

主存块号	块内地址
22位	6位



考点九：存储器

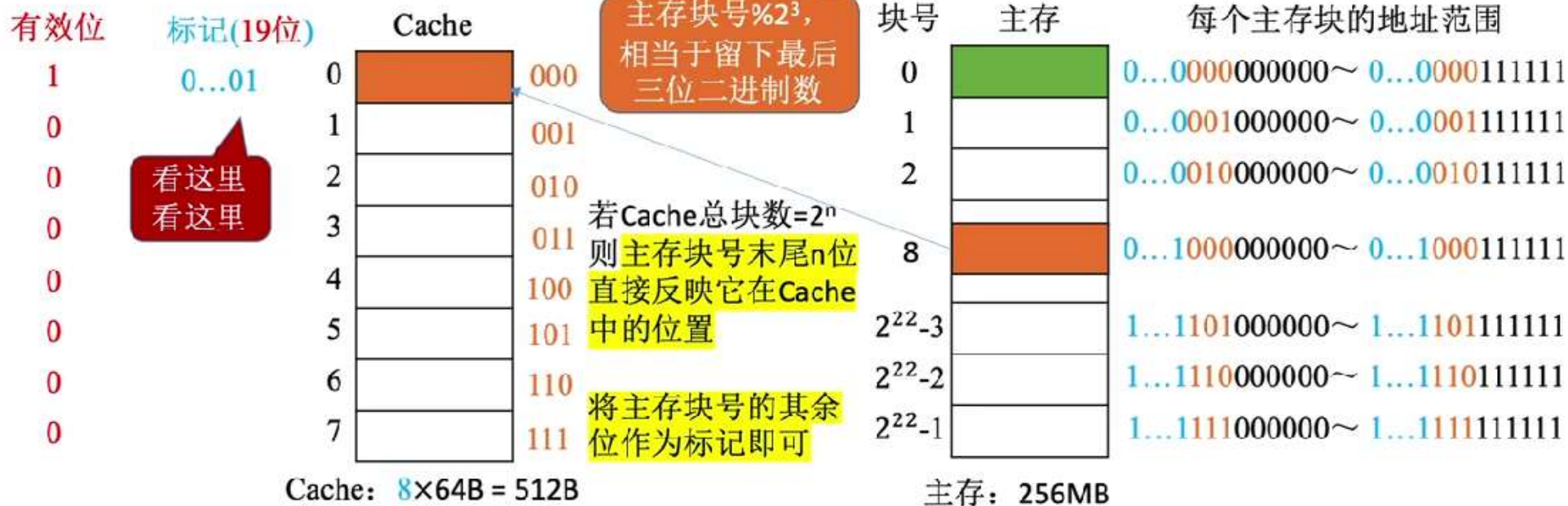
Cache——直接映射

假设某个计算机的主存地址空间大小为256MB，按字节编址，其数据Cache有8个Cache行，行长为64B。

直接映射，主存块在Cache中的位置=主存块号%Cache总块数

$256\text{M}=2^{28}$ 主存的地址共28位：

主存块号	块内地址
22位	6位



考点九：存储器

Cache——直接映射

假设某个计算机的主存地址空间大小为256MB，按字节编址，其数据Cache有8个Cache行，行长为64B。

直接映射，主存块在Cache中的位置=主存块号%Cache总块数

$256\text{M}=2^{28}$ 主存的地址共28位：

主存块号		块内地址
22位		6位
19位 标记	3位 行号	6位块内 地址

Cache 共 2^3 行

有效位	标记(19位)	Cache	CPU 访问主存地址 0...01000 001110 :	块号	主存	每个主存块的地址范围
1	0...01	0 000	①根据主存块号的后3位确定Cache行	0		0...0000000000~ 0...0000111111
0		1 001	②若主存块号的前19位与Cache标记匹配且有效位=1，则Cache命中，访问块内地址为001110的单元。	1		0...0001000000~ 0...0001111111
0		2 010	③若未命中或有效位=0，则正常访问主存	2		0...0010000000~ 0...0010111111
0		3 011		8		0...1000000000~ 0...1000111111
0		4 100		$2^{22}-3$		1...1101000000~ 1...1101111111
0		5 101		$2^{22}-2$		1...1110000000~ 1...1110111111
0		6 110		$2^{22}-1$		1...1111000000~ 1...1111111111
0		7 111				

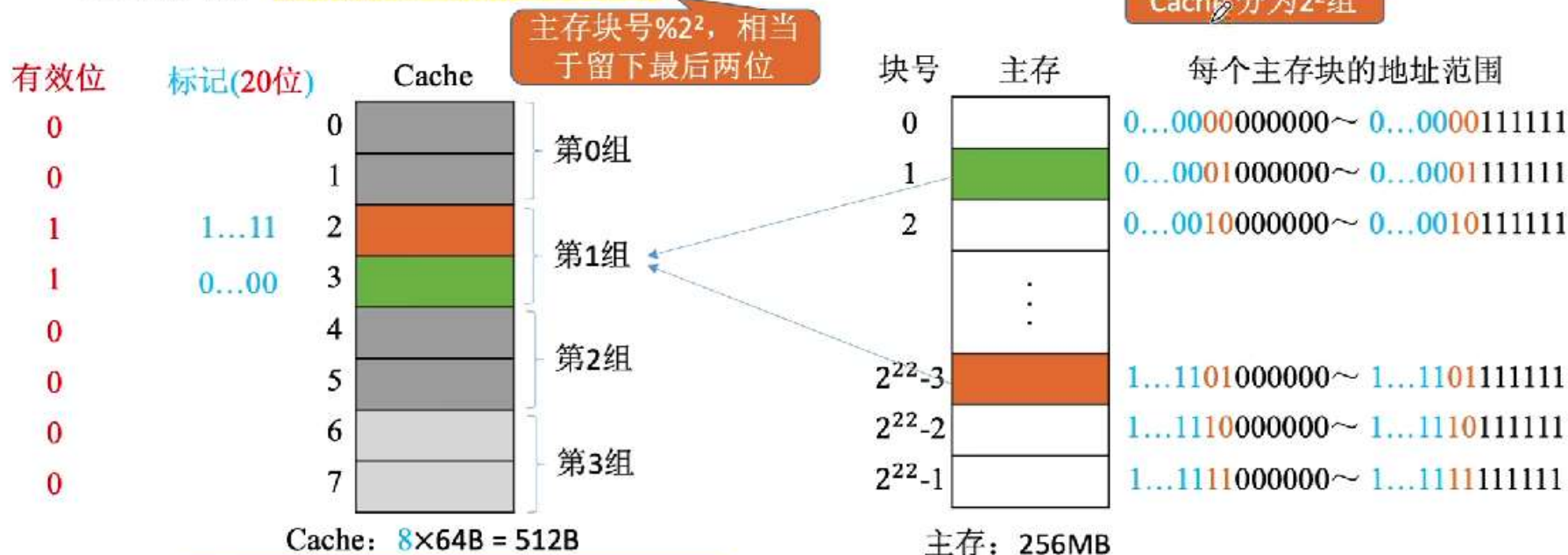
Cache: $8 \times 64\text{B} = 512\text{B}$ 主存: 256MB

考点九：存储器

Cache——组相联映射

假设某个计算机的主存地址空间大小为256MB，按字节编址，其数据Cache有8个Cache行，行长为64B。

组相联映射，所属分组=主存块号%分组数



2路组相联映射——2块为一组，分四组

256M=2²⁸ 主存的地址共28位:

主存块号		块内地址
22位		6位
20位 标记	2位 组号	6位块内 地址

Cache分为2²组

考点九：存储器

Cache——组相联映射

假设某个计算机的主存地址空间大小为256MB，按字节编址，其数据Cache有8个Cache行，行长为64B。

组相联映射，所属分组=主存块号%分组数

$256\text{M}=2^{28}$ 主存的地址共28位：

主存块号		块内地址
22位		6位
20位 标记	2位 组号	6位块内 地址

Cache 分为 2^2 组

有效位	标记(20位)	Cache	CPU 访问主存地址 1...1101001110 :	块号	主存	每个主存块的地址范围
0		0 00	①根据主存块号的后	0		0...0000000000~ 0...0000111111
0		1 00	2位确定所属分组号	1		0...0001000000~ 0...0001111111
1	1...11	2 01	②若主存块号的前20	2		0...0010000000~ 0...0010111111
1	0...00	3 01	位与分组内的某个标		⋮	
0		4 10	记匹配且有效位=1，		⋮	
0		5 10	则Cache命中，访问	$2^{22}-3$		1...1101000000~ 1...1101111111
0		6 11	块内地址为 001110	$2^{22}-2$		1...1110000000~ 1...1110111111
0		7 11	的单元。	$2^{22}-1$		1...1111000000~ 1...1111111111
0			③若未命中或有效位			
			=0，则正常访问主存			

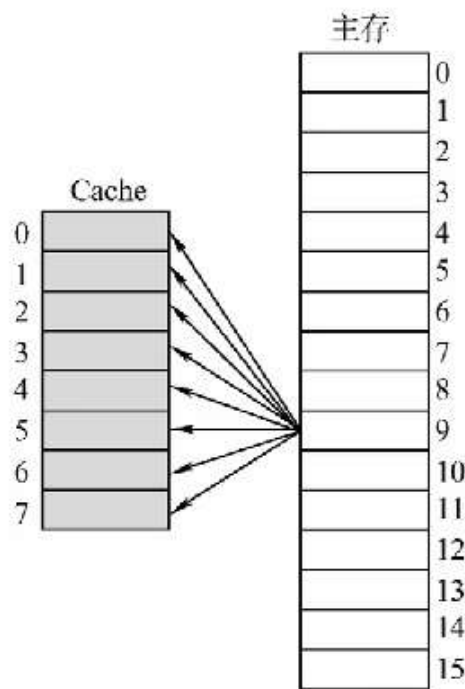
Cache: $8 \times 64\text{B} = 512\text{B}$

主存: 256MB

2路组相联映射——2块为一组，分四组

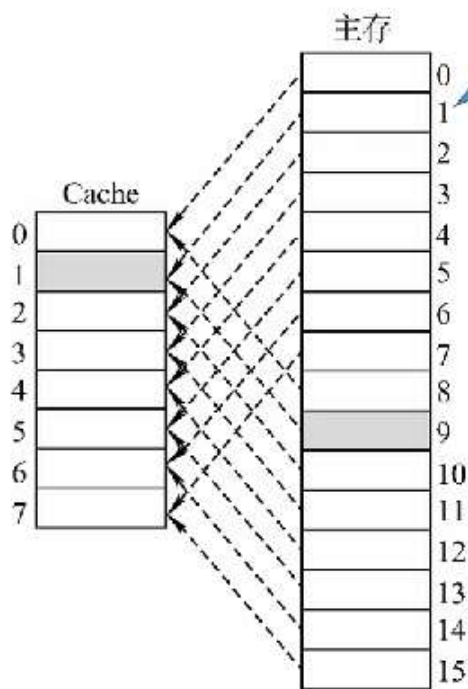
考点九：存储器

Cache——替换策略



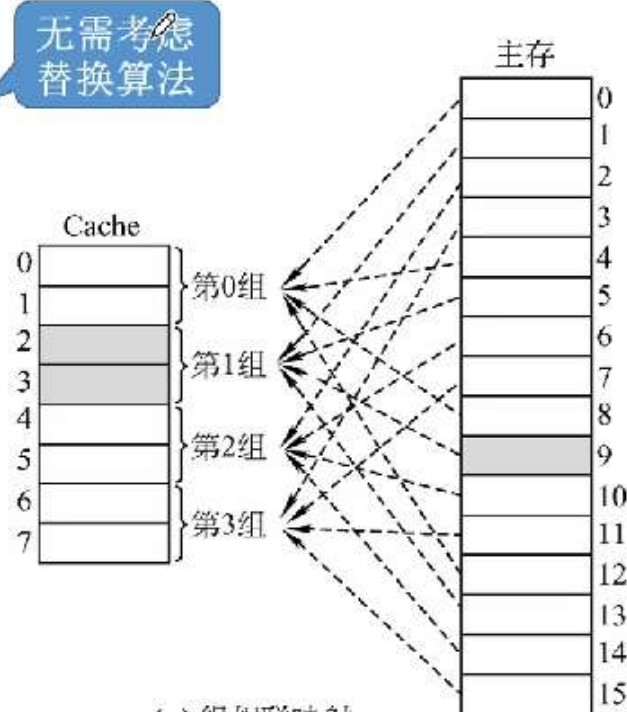
(a) 全相联映射

Cache完全满了才需要替换
需要**在全局**选择替换哪一块



(b) 直接映射

如果对应位置非空，则
毫无选择地直接替换



(c) 组相联映射

分组内满了才需要替换
需要在**分组内**选择替换哪一块

考点九：存储器

Cache替换策略——随机RAND

随机算法（**RAND**, Random）——若Cache已满，则随机选择一块替换。

设总共有 4 个Cache块，初始整个Cache为空。采用全相联映射，依次访问主存块 {1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5}



歪，你有freestyle吗？

访问主存块	1	2	3	4	1	2	5	1	2	3	4	5
Cache #0	1	1	1	1	1	1	1	1	1	1	4	4
Cache #1		2	2	2	2	2	2	2	2	2	2	2
Cache #2			3	3	3	3	5	5	5	5	5	5
Cache #3				4	4	4	4	4	4	3	3	3
Cache命中?	否	否	否	否	是	是	否	是	是	否	否	是
Cache替换?	否	否	否	否	否	否	是	否	否	是	是	否

随机算法——实现简单，但完全没考虑局部性原理，命中率低，实际效果很不稳定

考点九：存储器

Cache替换策略——先进先出FIFO

先进先出算法（**FIFO**, First In First Out）——若Cache已满，则替换最先被调入Cache的块

设总共有 4 个Cache块，初始整个Cache为空。采用全相联映射，依次访问主存块 {1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5}

访问主存块	1	2	3	4	1	2	5	1	2	3	4	5
Cache #0	1	1	1	1	1	1	5	5	5	5	4	4
Cache #1		2	2	2	2	2	2	1	1	1	1	5
Cache #2			3	3	3	3	3	3	2	2	2	2
Cache #3				4	4	4	4	4	4	3	3	3
Cache命中?	否	否	否	否	是	是	否	否	否	否	否	否
Cache替换?	否	否	否	否	否	否	是	是	是	是	是	是

先进先出算法——实现简单，最开始按#0#1#2#3放入Cache，之后轮流替换 #0#1#2#3
FIFO依然没考虑局部性原理，最先被调入Cache的块也有可能是被频繁访问的

考点九：存储器

Cache替换策略——最近最少使用LRU

近期最少使用算法（**LRU**, Least Recently Used）——为每一个Cache块设置一个“计数器”，用于记录每个Cache块已经有多久没被访问了。当Cache满后替换“计数器”最大的

设总共有4个Cache块，初始整个Cache为空。采用全相联映射，依次访问主存块 {1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5}

计数器	访问主存块	1	2	3	4	1	2	5	1	2	3	4	5
0	Cache #0	1											
0	Cache #1												
0	Cache #2												
0	Cache #3												
	Cache命中?	否											
	Cache替换?	否											

- ①命中时，所命中的行的计数器清零，比其低的计数器加1，其余不变；
- ②未命中且还有空闲行时，新装入的行的计数器置0，其余非空闲行全加1；
- ③未命中且无空闲行时，计数值最大的行的信息块被淘汰，新装行的块的计数器置0，其余全加1。

考点九：存储器

Cache替换策略——最近最少使用LRU

近期最少使用算法（LRU, Least Recently Used）——为每一个Cache块设置一个“计数器”，用于记录每个Cache块已经有多久没被访问了。当Cache满后替换“计数器”最大的

设总共有4个Cache块，初始整个Cache为空。采用全相联映射，依次访问主存块 {1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5}

计数器	访问主存块	1	2	3	4	1	2	5	1	2	3	4	5
1	Cache #0	1	1										
0	Cache #1		2										
0	Cache #2												
0	Cache #3												
	Cache命中?	否	否										
	Cache替换?	否	否										

- ①命中时，所命中的行的计数器清零，比其低的计数器加1，其余不变；
- ②未命中且还有空闲行时，新装入的行的计数器置0，其余非空闲行全加1；
- ③未命中且无空闲行时，计数值最大的行的信息块被淘汰，新装行的块的计数器置0，其余全加1。

考点九：存储器

Cache替换策略——最近最少使用LRU

近期最少使用算法 (LRU, Least Recently Used) —— 为每一个Cache块设置一个“计数器”，用于记录每个Cache块已经有多久没被访问了。当Cache满后替换“计数器”最大的

设总共有 4 个Cache块，初始整个Cache为空。采用全相联映射，依次访问主存块 {1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5}

计数器	访问主存块	1	2	3	4	1	2	5	1	2	3	4	5
3	Cache #0	1	1	1	1								
2	Cache #1		2	2	2								
1	Cache #2			3	3								
0	Cache #3				4								
	Cache命中?	否	否	否	否								
	Cache替换?	否	否	否	否								

- ①命中时，所命中的行的计数器清零，比其低的计数器加1，其余不变；
- ②未命中且还有空闲行时，新装入的行的计数器置0，其余非空闲行全加1；
- ③未命中且无空闲行时，计数值最大的行的信息块被淘汰，新装行的块的计数器置0，其余全加1。

考点九：存储器

Cache替换策略——最近最少使用LRU

近期最少使用算法（LRU, Least Recently Used）——为每一个Cache块设置一个“计数器”，用于记录每个Cache块已经有多久没被访问了。当Cache满后替换“计数器”最大的

设总共有4个Cache块，初始整个Cache为空。采用全相联映射，依次访问主存块 {1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5}

计数器		访问主存块	1	2	3	4	1	2	5	1	2	3	4	5
0	3	Cache #0	1	1	1	1	1							
3	2	Cache #1		2	2	2	2							
2	1	Cache #2			3	3	3							
1	0	Cache #3				4	4							
Cache命中?			否	否	否	否	是							
Cache替换?			否	否	否	否	否							

- ①命中时，所命中的行的计数器清零，比其低的计数器加1，其余不变；
- ②未命中且还有空闲行时，新装入的行的计数器置0，其余非空闲行全加1；
- ③未命中且无空闲行时，计数值最大的行的信息块被淘汰，新装行的块的计数器置0，其余全加1。

考点九：存储器

Cache替换策略——最近最少使用LRU

近期最少使用算法 (LRU, Least Recently Used) —— 为每一个Cache块设置一个“**计数器**”，用于记录每个Cache块已经有多久没被访问了。当Cache满后**替换“计数器”最大的**

设总共有 4 个Cache块，初始整个Cache为空。采用全相联映射，依次访问主存块 {1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5}

计数器		访问主存块	1	2	3	4	1	2	5	1	2	3	4	5
0	← 3	Cache #0	1	1	1	1	1	1	1	1	1	1	1	5
3	2	Cache #1		2	2	2	2	2	2	2	2	2	2	2
1	0	Cache #2			3	3	3	3	5	5	5	5	4	4
2	1	Cache #3				4	4	4	4	4	4	3	3	3
Cache命中?			否	否	否	否	是	是	否	是	是	否	否	否
Cache替换?			否	否	否	否	否	否	是	否	否	是	是	是

Cache块的总数 $= 2^n$ ，则计数器只需n位。且Cache装满后所有计数器的值一定不重复

- ①命中时，所命中的行的计数器清零，**比其低的计数器加1**，其余不变；
- ②未命中且还有空闲行时，新装入的行的计数器置0，其余非空闲行全加1；
- ③未命中且无空闲行时，计数值最大的行的信息块被淘汰，新装行的块的计数器置0，其余全加1。

考点九：存储器 Cache替换策略——最不常使用LFU

最不经常使用算法（**LFU**, Least Frequently Used）——为每一个Cache块设置一个“**计数器**”，用于记录每个Cache块被访问过几次。当Cache满后替换“**计数器**”最小的

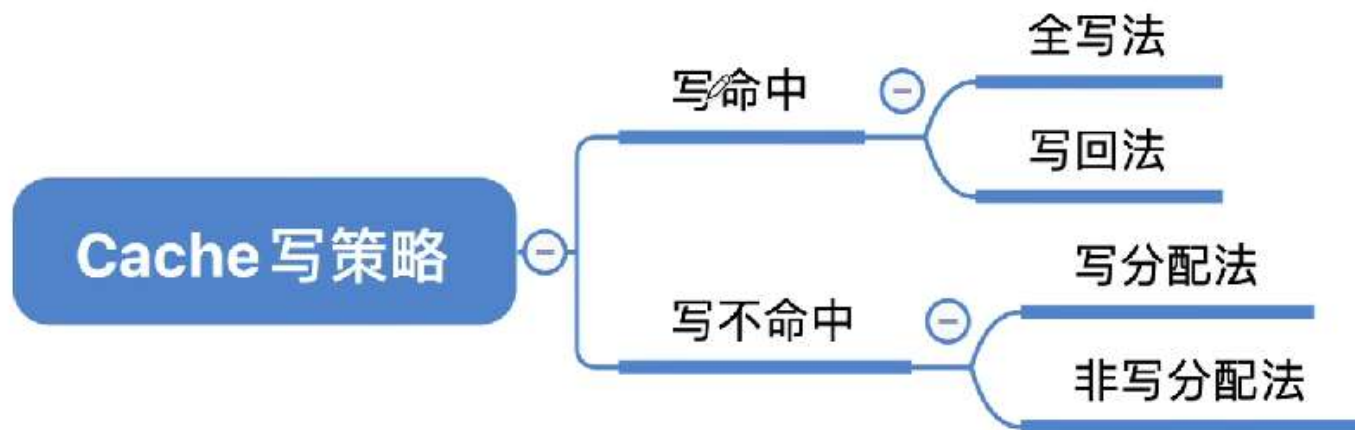
设总共有 4 个Cache块，初始整个Cache为空。采用全相联映射，依次访问主存块 {1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5}

计数器	访问主存块	1	2	3	4	1	2	5	1	2	3	4	5
2	Cache #0	1	1	1	1	1	1	1	1	1	1	1	1
2	Cache #1		2	2	2	2	2	2	2	2	2	2	2
0	Cache #2			3	3	3	3	5	5	5	3	3	5
1	Cache #3				4	4	4	4	4	4	4	4	4
	Cache命中?	否	否	否	否	是	是	否	是	是	否	是	否
	Cache替换?	否	否	否	否	否	否	是	否	否	是	否	是

LFU算法——曾经被经常访问的主存块在未来不一定会用到（如：微信视频聊天相关的块），并没有很好地遵循局部性原理，因此实际运行效果不如LRU

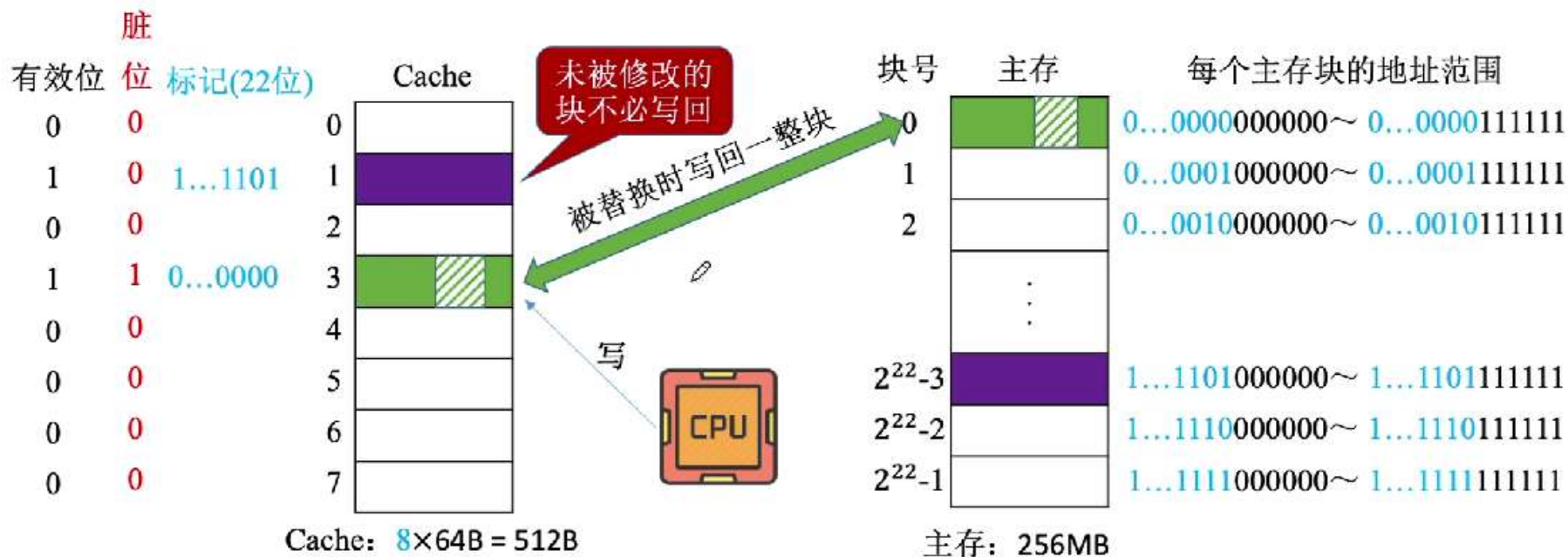
考点九：存储器

Cache——写策略



考点九：存储器

Cache——写命中

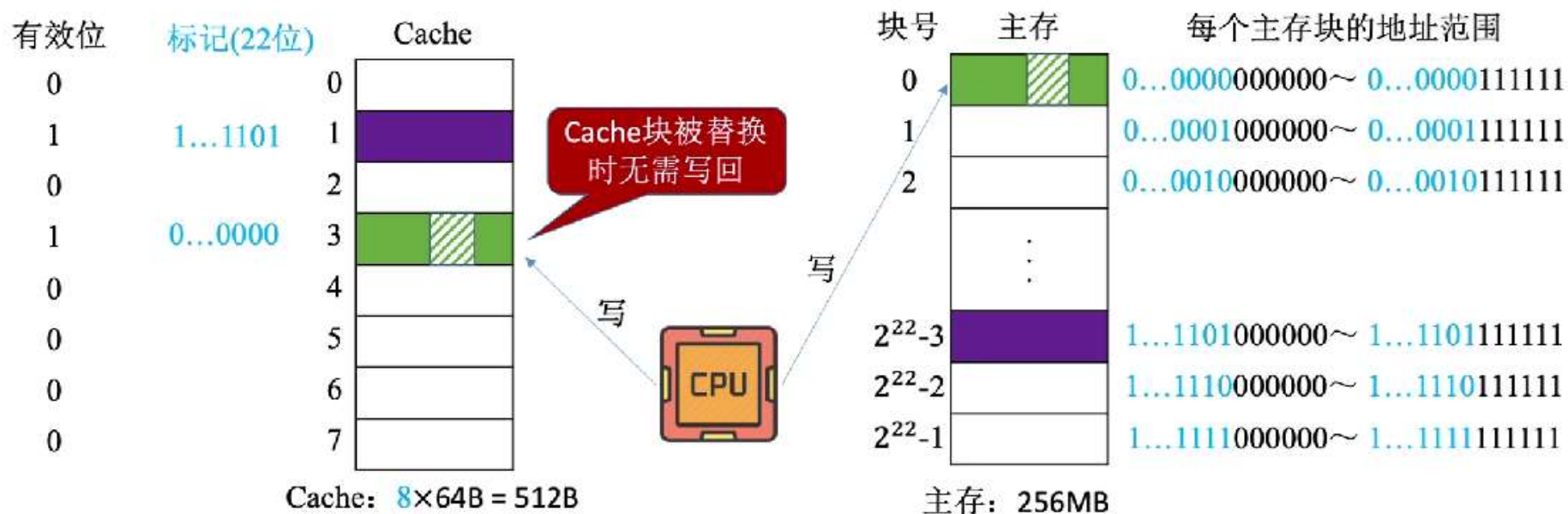


写回法(write-back)——当CPU对Cache写命中时，只修改Cache的内容，而不立即写入主存，只有当此块被换出时才写回主存

减少了访存次数，但存在数据不一致的隐患。

考点九：存储器

Cache——写命中

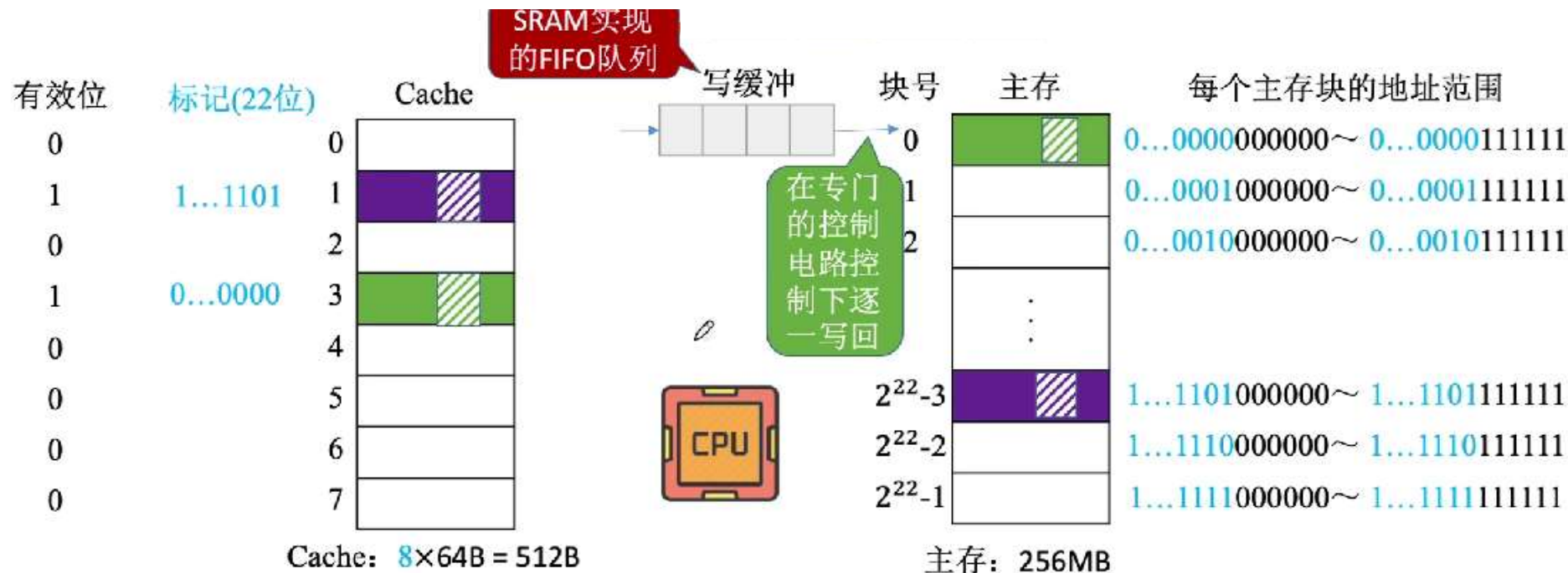


全写法(写直通法, write-through) —— 当CPU对Cache写命中时, 必须把数据同时写入Cache和主存, 一般使用写缓冲(write buffer)

访存次数增加, 速度变慢, 但更能保证数据一致性

考点九：存储器

Cache——写命中

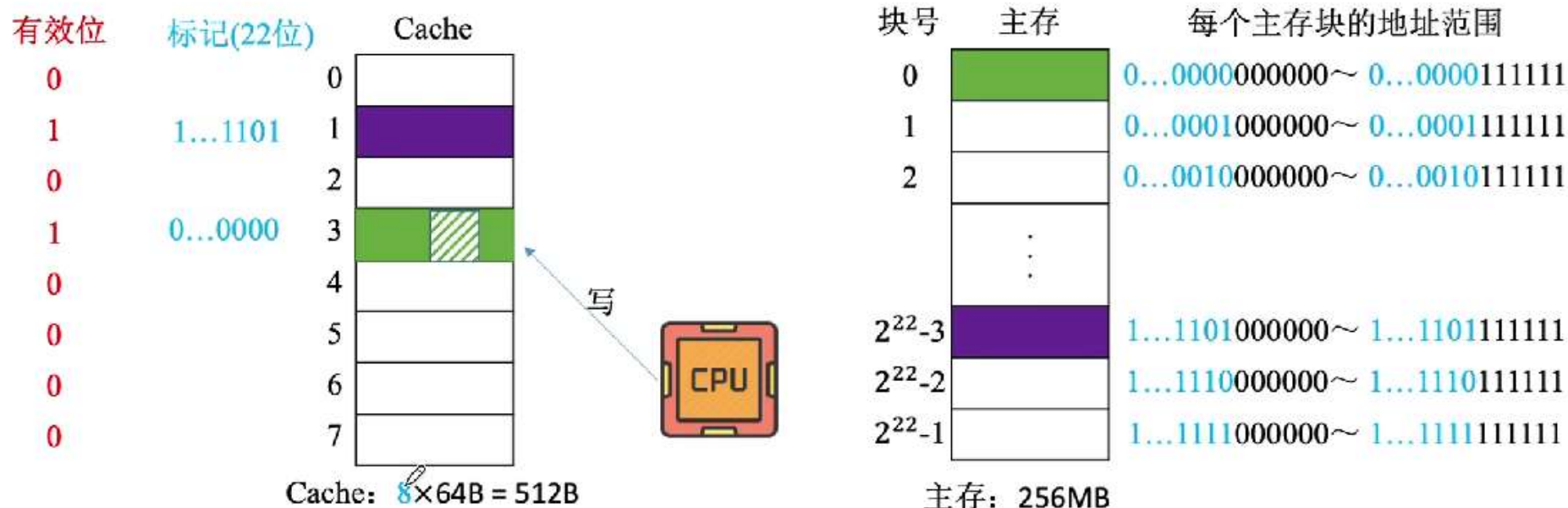


全写法(写直通法, write-through) —— 当CPU对Cache写命中时, 必须把数据同时写入Cache和主存, 一般使用写缓冲(write buffer)

使用写缓冲, CPU写的速度很快, 若写操作不频繁, 则效果很好。若写操作很频繁, 可能会因为写缓冲饱和而发生阻塞

考点九：存储器

Cache——写不命中



写分配法(write-allocate)——当CPU对Cache写不命中时，把主存中的块调入Cache，在Cache中修改。通常搭配写回法使用。

写回法(write-back)——当CPU对Cache写命中时，只修改Cache的内容，而不立即写入主存，只有当此块被换出时才写回主存

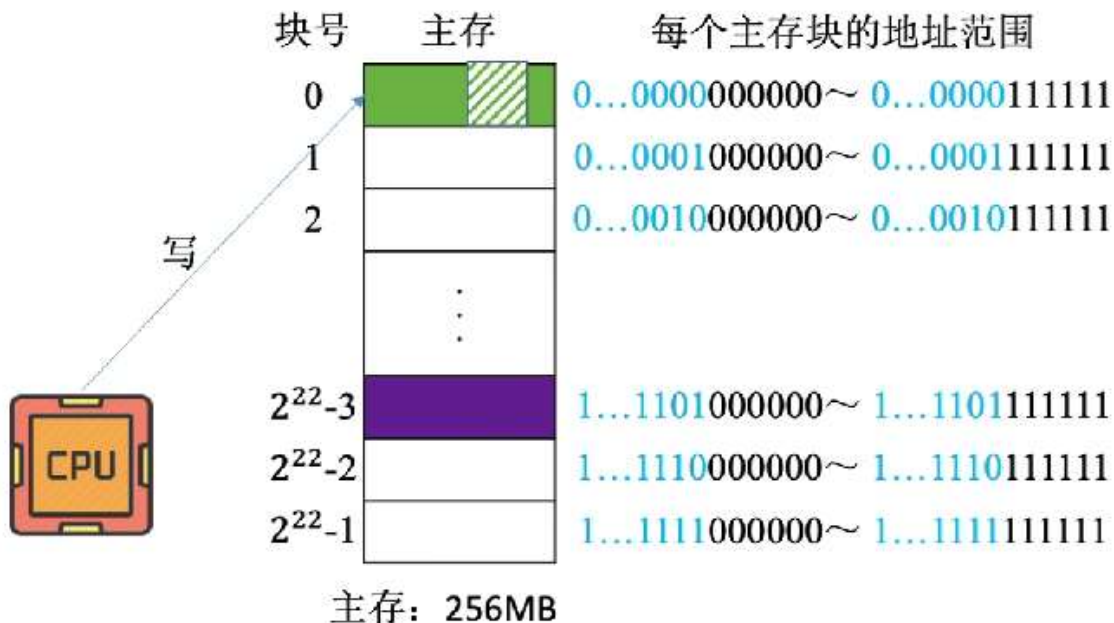
考点九：存储器

Cache——写不命中



只有“读”未命中时才调入Cache

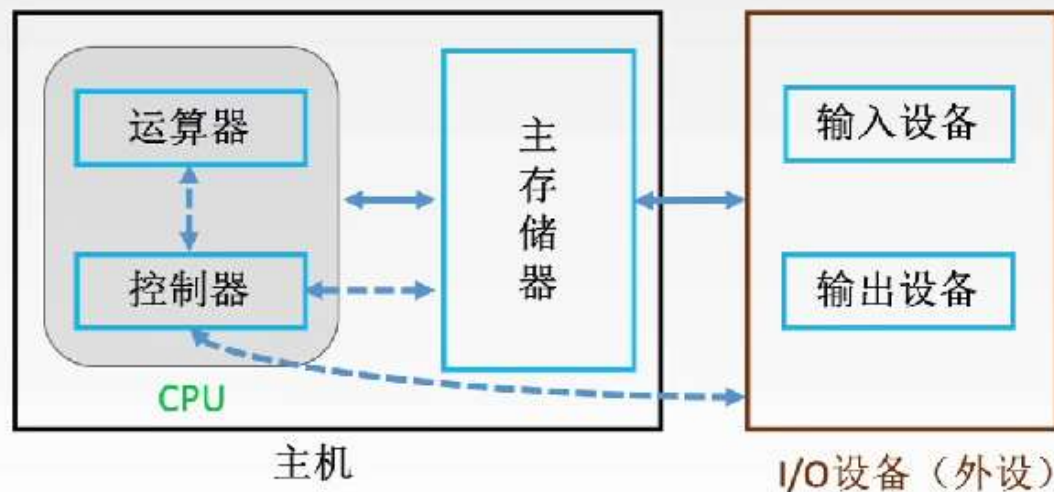
Cache: $8 \times 64\text{B} = 512\text{B}$



非写分配法(not-write-allocate)——当CPU对Cache写不命中时只写入主存，不调入Cache。
搭配全写法使用。

全写法(写直通法, write-through)——当CPU对Cache写命中时，必须把数据同时写入Cache和主存，一般使用写缓冲(write buffer)

考点十：IO

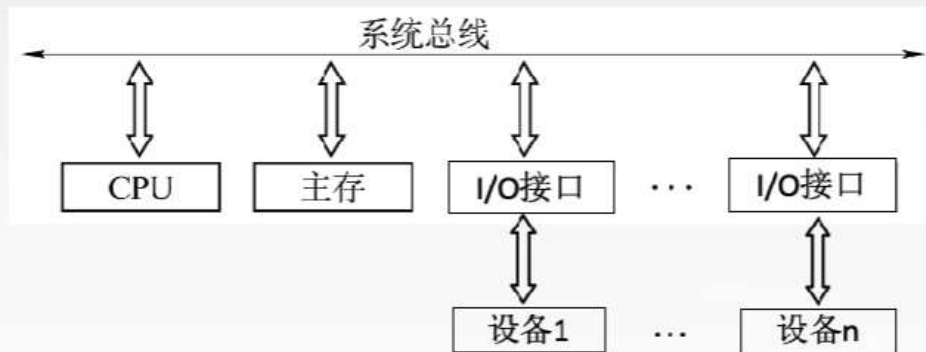


“I/O” 就是 “输入/输出” (Input/Output)

I/O 设备就是可以将数据输入到计算机，或者可以接收计算机输出数据的外部设备

考点十：IO

IO接口



I/O接口：又称**I/O控制器**（**I/O Controller**）、**设备控制器**，负责协调主机与外部设备之间的数据传输

考点十：IO

IO接口——作用



- 数据缓冲：通过数据缓冲寄存器（DBR）达到主机和外设工作速度的匹配
- 错误或状态监测：通过状态寄存器反馈设备的各种错误、状态信息，供CPU查用
- 控制和定时：接收从控制总线发来的控制信号、时钟信号
- 数据格式转换：串-并、并-串 等格式转换
- 与主机和设备通信：实现 主机—I/O接口—I/O设备 之间的通信

考点十：IO

IO接口——编址方式

1. 统一编址

把I/O端口当做存储器的单元进行地址分配，用统一的**访存指令**就可以访问I/O端口，又称**存储器映射方式**。

靠不同的**地址码**区分内存和I/O设备，I/O地址要求相对固定在地址的某部分。

优点：

不需要专门的输入/输出指令，所有访存指令都可直接访问端口，程序设计灵活性高
端口有较大的编址空间
读写控制逻辑电路简单

缺点：

端口占用了主存地址空间，使主存地址空间变小
外设寻址时间长（地址位数多，地址译码速度慢）

2. 独立编址

I/O端口地址与存储器地址无关，独立编址CPU需要设置**专门的输入/输出指令**访问端口，又称**I/O映射方式**。

靠不同的**指令**区分内存和I/O设备。

优点：

使用专用I/O指令，程序编制清晰
I/O端口地址位数少，地址译码速度快
I/O端口的地址不占用主存地址空间

缺点：

I/O指令类型少，一般只能对端口进行传送操作，程序设计灵活性差
需要CPU提供存储器读/写、I/O设备读/写两组控制信号，增加了控制逻辑电路的复杂性

考点十：IO

IO接口——分类

按数据传送方式可分为

并行接口：一个字节或一个字所有位同时传送。

串行接口：一位一位地传送。

注：这里所说的数据传送方式指的是外设和接口一侧的传送方式，而在主机和接口一侧，数据总是并行传送的。接口要完成数据格式转换。

按主机访问I/O设备的控制方式可分为

程序查询接口

中断接口

DMA接口

按功能选择的灵活性可分为

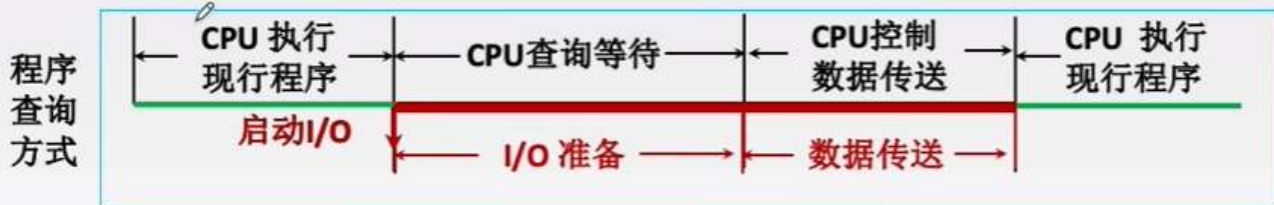
可编程接口

不可编程接口

考点十：IO

IO控制方式——程序查询

1) **程序查询方式**：CPU不断轮询检查I/O控制器中的“状态寄存器”，检测到状态为“已完成”之后，再从数据寄存器取出输入数据



CPU一旦启动I/O，必须停止现程序的运行，并在现程序中插入一段程序。

主要特点：CPU有“踏步”等待现象，CPU与I/O串行工作。

优点：接口设计简单、设备量少。

缺点：CPU在信息传送过程中要花费很多时间用于查询和等待，而且如果采用独占查询，则在一段时间内只能和一台外设交换信息，效率大大降低。

独占查询：CPU 100%的时间都在查询I/O状态，完全串行

定时查询：在保证数据不丢失的情况下，每隔一段时间CPU就查询一次I/O状态。查询的间隔内CPU可以执行其他程序

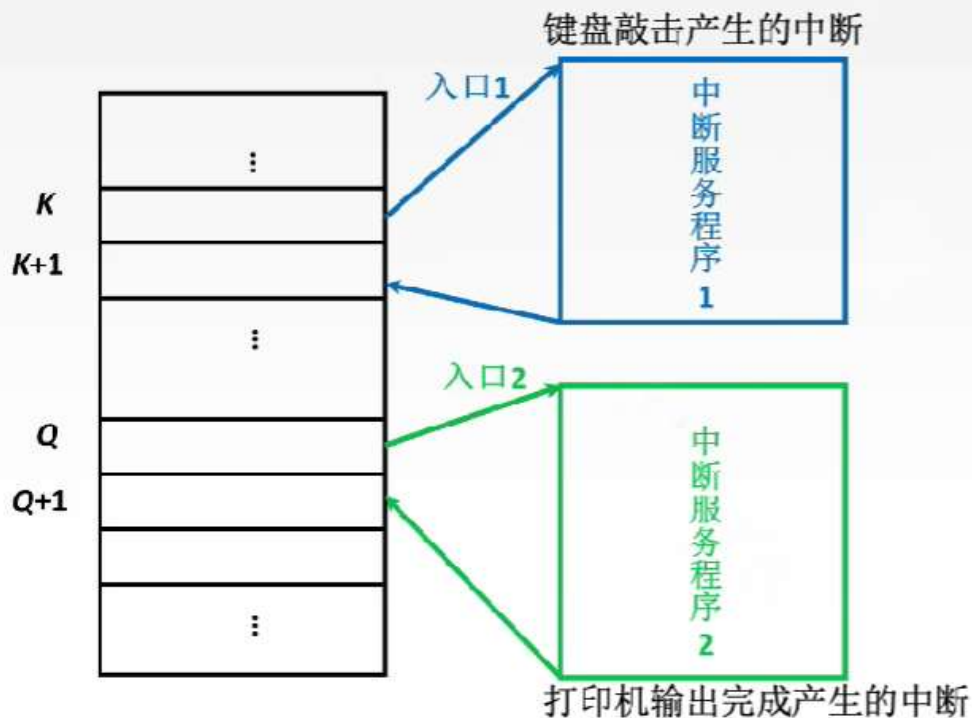


我怎么会骗你呢

考点十：IO

IO控制方式——程序中断

程序中断是指在计算机执行现行程序的过程中，出现某些急需处理的异常情况或特殊请求，CPU暂时中止现行程序，而转去对这些异常情况或特殊请求进行处理，在处理完毕后CPU又自动返回到现行程序的断点处，继续执行原程序。

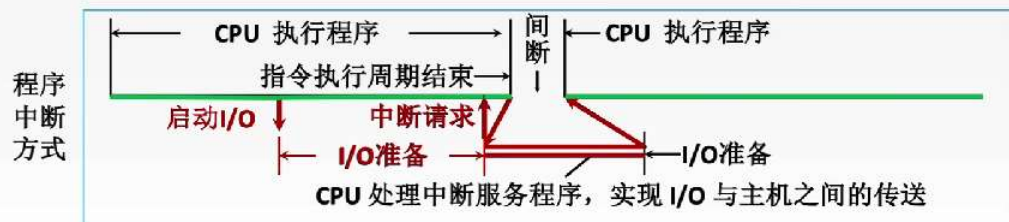
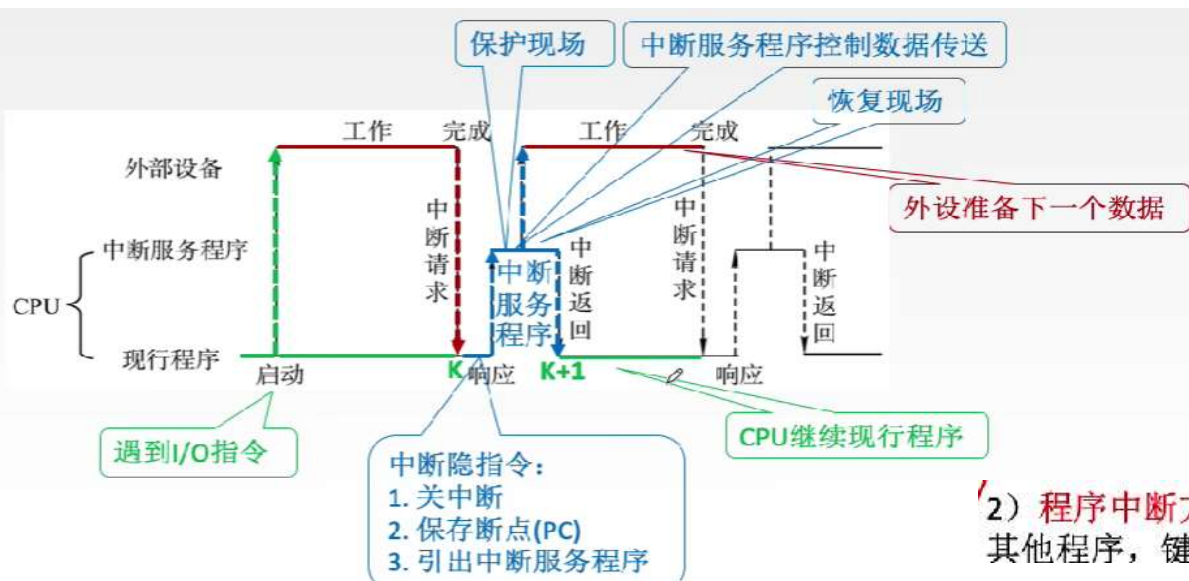


工作流程：

1. 中断请求
中断源向CPU发送中断请求信号。
2. 中断响应
响应中断的条件。
中断判优：多个中断源同时提出请求时通过中断判优逻辑响应一个中断源。
3. 中断处理
中断隐指令。
中断服务程序。

考点十：IO

IO控制方式——程序中中断

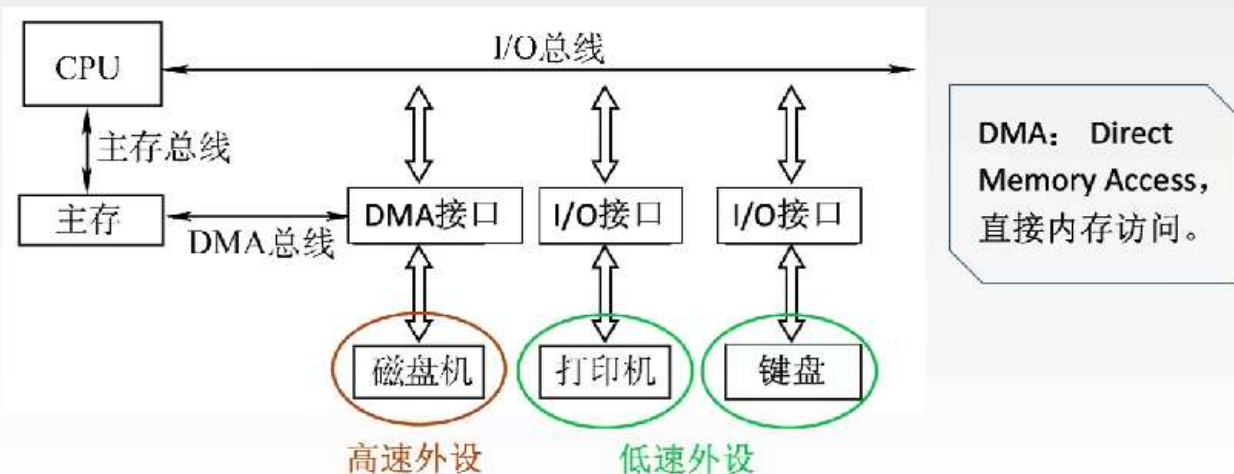


思考：对于快速I/O设备，如“磁盘”，每准备好一个字就给CPU发送一次中断请求，会导致什么问题？

答：CPU需要花大量的时间来处理中断服务程序，CPU利用率严重下降。

考点十：IO

IO控制方式——DMA



注：DMA接口，即DMA控制器，也是一种特殊的I/O控制器

DMA控制方式：主存与高速I/O设备之间有一条**直接数据通路**（DMA总线）。CPU向DMA接口发出“读/写”命令，并指明主存地址、磁盘地址、读写数据量等参数。DMA控制器自动控制磁盘与主存的数据读写，**每完成一整块数据读写**（如1KB为一整块），**才向CPU发出一次中断请求。**

考点十：IO

IO控制方式

	中断	DMA
数据传送	程序控制 程序的切换 → 保存和恢复现场	硬件控制 CPU只需进行预处理和后处理
中断请求	传送数据	后处理
响应	指令执行周期结束后响应中断	每个机器周期结束均可，总线空闲时即可响应DMA请求
场景	CPU控制，低速设备	DMA控制器控制，高速设备
优先级	优先级低于DMA	优先级高于中断 ∅
异常处理	能处理异常事件	仅传送数据